

Παρουσίαση πάνω στην τέταρτη εργασία

#####

AEM: ####

21/1/2025

Μέρος 1ο: Αποστολή δεδομένων στο εικονικό τερματικό μέσω UART

Στο πρώτο μέρος της εργασίας θα χρειαστεί να στείλουμε δεδομένα απο την πλακέτα σε εικονικό τερματικό, καθώς και να υλοποιήσουμε μία μονάδα που θα μετατρέπει τιμές binary σε αντίστοιχες τιμές ascii.

- Οι μονάδες υπεύθυνες για την αποστολή ascii τιμών μέσω uart είναι ήδη υλοποιημένες. Για την δοκιμή θα χρησιμοποιήσουμε τον πελάτη σειριακής επικοινωνίας PuTTY, ρυθμίζοντας την ταχύτητα λύψης Baud σε 57600. Ανα μετάδοση θα στέλνουμε ένα byte με δεδομένα, καθώς και ένα parity bit (θα χρησιμοποιήσουμε odd parity).
- Κατα την δοκιμή με την χρήση της μονάδας demo δεν υπήρξε πρόβλημα στην μεταβολή και την λήψη, οπότε παρακάτω θα μελετηθεί το δεύτερο κομμάτι αυτού του μέρους .

Μετατροπή δεδομένων απο binary σε bcd

- Πρώτο βήμα της μετατροπής απο binary σε ascii είναι η μετατροπή των δεδομένων σε bcd. Θα υλοποιήσουμε αυτή τη λειτουργία χρησιμοποιώντας τον αλγόριθμο “Double-Daddle”
- Ο αλγόριθμος λειτουργεί ως εξής: Για κάθε bit του αριθμού που εισάγουμε, θα ελένξουμε πρώτα αν κάποιο (4-bit) ψηφίο του bcd είναι μεγαλύτερο του 5. Αν είναι, θα προσθέσουμε 3 στο αντίστοιχο ψηφίο του bcd. Σε κάθε περίπτωση, κάνουμε shift τον αριθμό κατα 1.
- Στη verilog αυτός θα υλοποιηθεί με ένα always block συνδιαστικής λογικής. Αφού ο αριθμός είναι 17 bit σε binary, μας αρκούν 24 bit για να τον αποθηκεύσουμε σε bcd.

0000	0000	0000	11110011	Initialization
0000	0000	0001	11100110	Shift
0000	0000	0011	11001100	Shift
0000	0000	0111	10011000	Shift
0000	0000	1010	10011000	Add 3 to ONES, since it was 7
0000	0001	0101	00110000	Shift
0000	0001	1000	00110000	Add 3 to ONES, since it was 5
0000	0011	0000	01100000	Shift
0000	0110	0000	11000000	Shift
0000	1001	0000	11000000	Add 3 to TENS, since it was 6
0001	0010	0001	10000000	Shift
0010	0100	0011	00000000	Shift
2	4	3		
BCD				

Εικόνα 1-1: Παράδειγμα εφαρμογής του αλγορίθμου double-daddle

Μετατροπή δεδομένων απο bcd σε ascii

13	CR	45	-
14	SO	46	.
15	SI	47	/
16	DLE	48	0
17	DC1	49	1
18	DC2	50	2
19	DC3	51	3
20	DC4	52	4
21	NAK	53	5
22	SYN	54	6
23	ETB	55	7
24	CAN	56	8
25	EM	57	9
26	SUB	58	:
27	ESC	59	:

- Για την μετατροπή των δεδομένων απο bcd σε ascii, αρκεί να προσθέσουμε 48 σε κάθε ψηφίο του bcd.
- Σε ascii, οι αριθμοί απο 0-9 αντιπροσωπεύονται με τους αντίστοιχους αριθμούς 48-57. Για αυτό η πρόσθεση με το 48 θα οδηγήσει σε σωστή αναπαράσταση
- Ένα ψηφίο στο bcd αναπαριστάται απο 4 bits, άρα για την υλοποίηση του μετατροπέα σε verilog, αρκεί ένα συνδιαστικό always block που αρθροίζει ανα τέσσερα τα ψηφια του bcd και τα αποθηκεύει σε μια ξεχωριστή μεταβλητή

Εικόνα 1-2: Αναπαράσταση Αριθμών σε ASCII

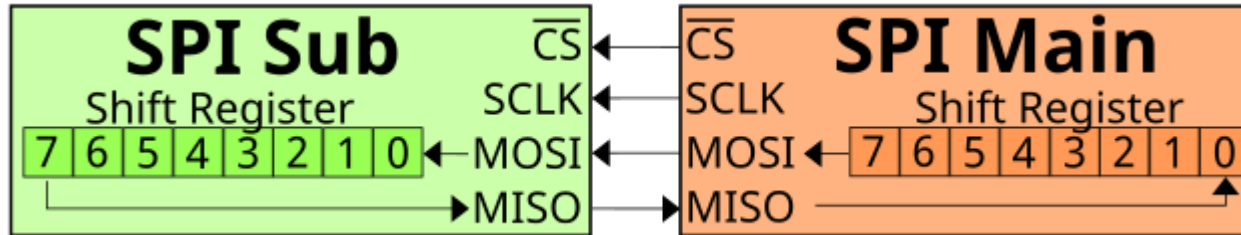
Πλαίσιο Δοκιμής

- Για τον έλεγχο ορθής λειτουργίας, αρκεί η δοκιμή τυχαίων τιμών
- Οι κυματομορφές είναι σε post implementation
- Η έξοδο της κονσόλας παρέχονται απο το testbench

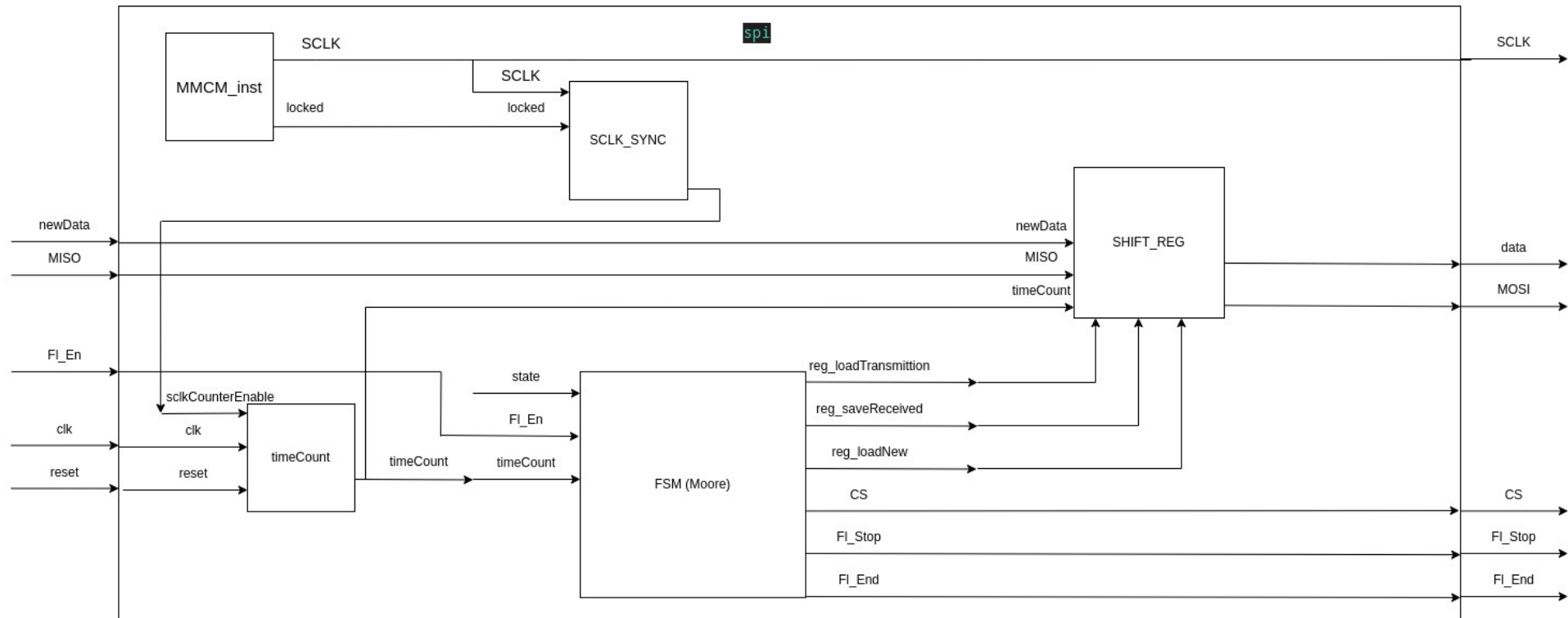
> bin[16:0]	1234	0	1	9	10	99	1234	99999	0
> ascii[47:0]	001234	000000	000001	000009	000010	000099	001234	099999	000000

```
# run 1000ns
Testing bin2ascii...
Binary:    0, ASCII: 000000
Binary:    1, ASCII: 000001
Binary:    9, ASCII: 000009
Binary:   10, ASCII: 000010
Binary:   99, ASCII: 000099
Binary:  1234, ASCII: 001234
Binary:  99999, ASCII: 099999
Binary:    0, ASCII: 000000
Test completed.
```

Μέρος 2ο: Υλοποίηση Ελεγκτή SPI Master

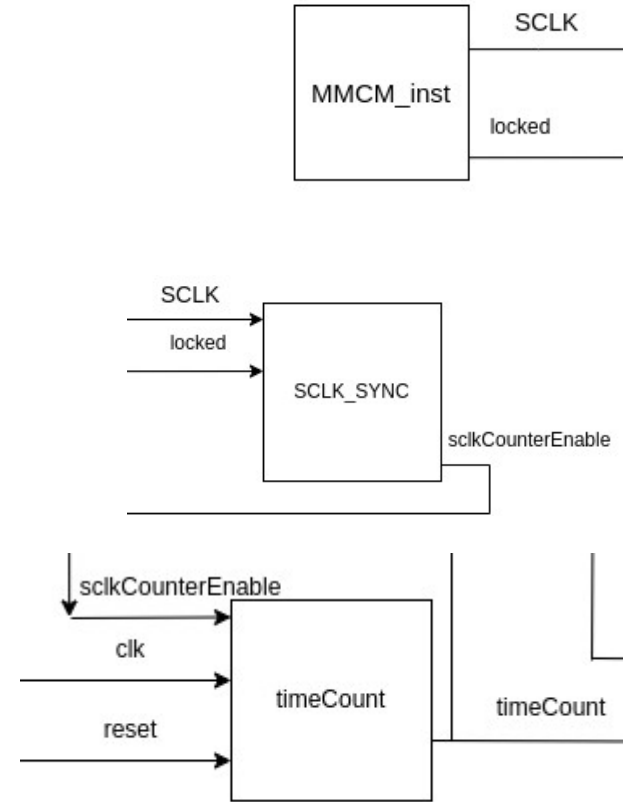


- Θα επικοινωνήσουμε με το αξελερόμετρο χρησιμοποιώντας την διεπαφή SPI. Για αυτό θα χρειαστεί να υλοποιήσουμε μια μονάδα-ελεγκτή SPI Master.
- Για την σωστή επικοινωνία με το αξελερόμετρο, θα έχουμε $CPOL = CPHA = 0$ όπως και το ίδιο.
- Στην επόμενη διαφάνεια φαίνεται το διάγραμμα ροής για τον ελεγκτή SPI:



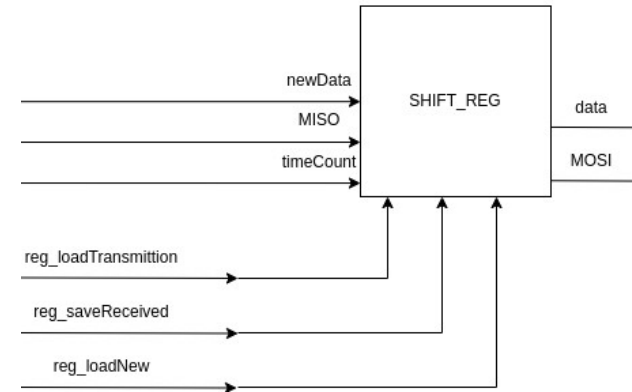
Αργό ρολόι και μετρητής

- Για να δημιουργήσουμε το αργό ρολόι που θα χρειαστεί το spi του αξελερόμετρου μπορούμε να χρησιμοποιήσουμε μια μονάδα MMCM
- Χρησιμοποιούμε ένα ρολόι 5MHz γιατί πληρεί τις προδιαγραφές και διαιρείται με το 100, μπορούμε δηλαδή να μετρήσουμε τους κύκλους του με το αργό ρολόι.
- Θα χρησιμοποιήσουμε έναν μετρητή για να επιτύχουμε το παραπάνω. Θα αρχίσει στη θετική ακμή του αργού ρολογιού sclk, όταν αυτό είναι συγχρονισμένο με το γρήγορο ρολόι.
- Για τον σωστό συγχρονισμό μεταξύ του μετρητή και των ρολογιών, θα χρησιμοποιήσουμε δυο συνδιαστικά always block, το πρώτο εκ των οποίων θα λειτουργεί με τη θετική ακμή του αργού ρολογιού. Στη πρώτη θετική ακμή του αργού ρολογιού όπου το mmcm είναι locked, θα σταλθεί ένα σήμα στο δεύτερο always block. Αυτό με τη σειρά του, όταν το εντοπίσει (στη θετική ακμή του γρήγορου ρολογιού), θα στείλει ένα σήμα στον μετρητή που θα τον επιτρέψει να “ακολουθήσει” το αργό ρολόι



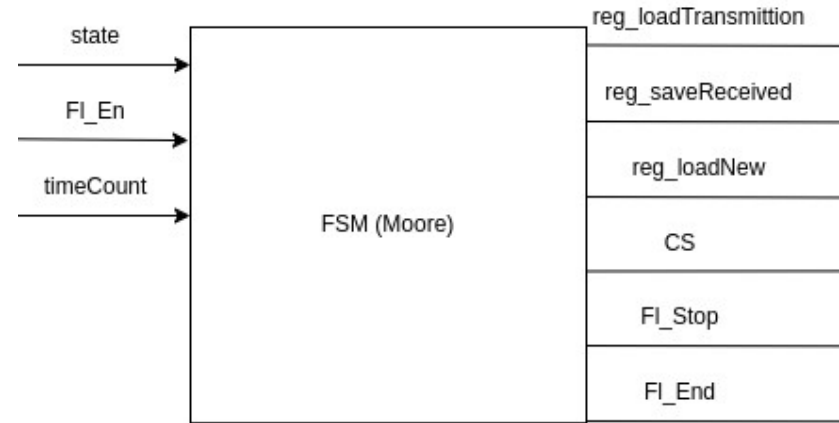
Shift Register

- Για την αποθήκευση των δεδομένων που θα ληφθούν, καθώς και αυτών που θα σταλθούν, μπορούμε να υλοποιήσουμε έναν shift register.
- Κάθε καινούργιο bit προς αποστολή θα αποτελεί το πιο σημαντικό bit του καταχωρητή.
- Κάθε καινούριο δεδομένο που θα λάβουμε θα εισαχθεί στο λιγότερο σημαντικό bit, αφού πρώτα μετακινήσουμε κάθε δεδομένα του καταχωρητή μία θέση αριστερά για να μη χάσουμε πληροφορία.
- Αυτό θα το υλοποιήσουμε με την χρήση δύο συνδιαστικών always blocks. Ένα θα αναθέτει στο mosi το πιο σημαντικό bit του καταχωρητή όταν ανεβεί το σήμα reg_loadTransmission, ενώ το άλλο θα κάνει shift τον καταχωρητή και θα αλλάζει το λιγότερο σημαντικό του bit όταν ανεβεί το σήμα reg_saveReceived. Μπορεί επίσης να θέσει τα δεδομένα του ίσα με newData όταν ανεβεί το σήμα reg_loadNew.
- Χρησιμοποιούμε τον μετρητή για να βεβαιωθούμε ότι η αλλαγή θα γίνει για μόνο έναν κύκλο
- Τα σήματα reg_loadTransmission, reg_saveReceived, reg_loadNew ελέγχονται από την FSM, λεπτομέρειες για την οποία φαίνονται παρακάτω



FSM

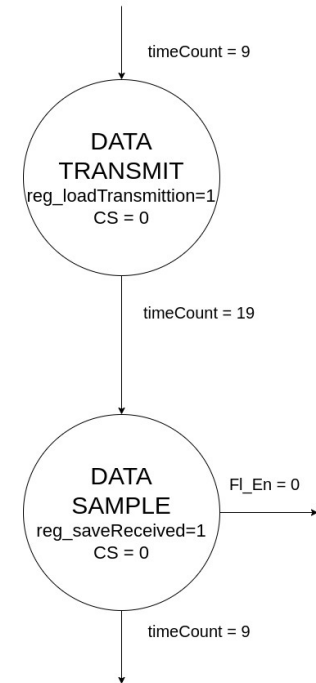
- Η σωστή ανάθεση των δεδομένων, ή καλύτερα η ανάθεση αυτών την σωστή χρονική στιγμή γίνεται με την χρήση FSM.
- Η FSM είναι τύπου moore και υλοποιήθηκε με τη χρήση 2 always block, ενός συνδιαστικού που θα είναι υπεύθυνο για τις εξόδους της (προφανώς αυτό περιλαμβάνει και την ανάθεση του επόμενου state), και ενός ακολουθιακού, που αναθέτει συνεχώς στο state το επόμενο state (που αναφέρθηκε προηγουμένως).
- Στις επόμενες διαφάνειες φαίνονται αναλυτικότερα τα state που περιέχει και την λειτουργία του καθενός απο αυτά.





FSM: States

- **IDLE:** Σε αυτό το state θα περιμένει ο ελεγκτής SPI μέχρι να σηματοδοτήσει ο οδηγός νέα αναμετάδοση, σηκώνοντας το FI_En. Αυτή είναι η αρχική κατάσταση, και εκεί που θα επιστρέψει ο οδηγός μετά το τέλος μιας μετάδοσης, ή μετά απο reset. Θα μεταβεί στο DATA PREPARE μετά τη σηματοδότηση αναμετάδοσης, ένα κύκλο πριν την αρνητική ακμή του SCLK.
- **DATA PREPARE:** Το state στο οποίο ο shift reg θα “φορτώσει” καινούρια δεδομένα απο τον οδηγό. Συγκεκριμένα, τα δεδομένα του καταχωρητή data (ο 8-bit shift register) θα αντικαθιστούνται απο τα δεδομένα του καλωδίου newData. Θα μεταβεί στο DATA1 TRANSMIT στην αρνητική ακμή του SCLK.
- **{DATAX TRANSMIT}:** Συμβολικό state που έχει την ίδια λογική με όλες τις καταστάσεις όμοιας ονομασίας. Σε αυτά τα state, το MOSI θα πάρει την τιμή του πιο σημαντικού bit του data. Η μηχανή θα μεταβεί στο {DATAX SAMPLE} στον θετικό παλμό του SCLK.
- **{DATAX SAMPLE}:** Συμβολικό state που έχει την ίδια λογική με όλες τις καταστάσεις όμοιας ονομασίας. Σε αυτό το state θα σηματοδοτηθεί στον καταχωρητή data να κάνει shift. Η μηχανή θα μεταβεί είτε στο STOP αν πέσει το FI_En. Αν δειγματοληπτούμε το τελευταίο (8ο) bit, θα ανεβεί το σήμα FI_End και η μηχανή θα μεταβεί στο DATA PREPARE ένα κύκλο πριν την αρνητική ακμή του SCLK. Αλλιώς θα μεταβεί στο αντίστοιχο {DATA(X+1) TRANSMIT} στην αρνητική ακμή του SCLK.
- **STOP:** Η μηχανή θα μεταβεί στο IDLE, περιμένοντας μέχρι την θετική ακμή του SCLK. Κατά τη διάρκεια αυτή θα είναι σηκωμένο το σήμα FI_Stop.



Πλαίσιο Δοκιμής

Για να επαληθεύσουμε τη σωστή λειτουργία της μονάδας, αρχικά πρέπει να επαληθεύσουμε ότι τα ρολόγια και ο μετρητής συγχρονίζονται σωστά

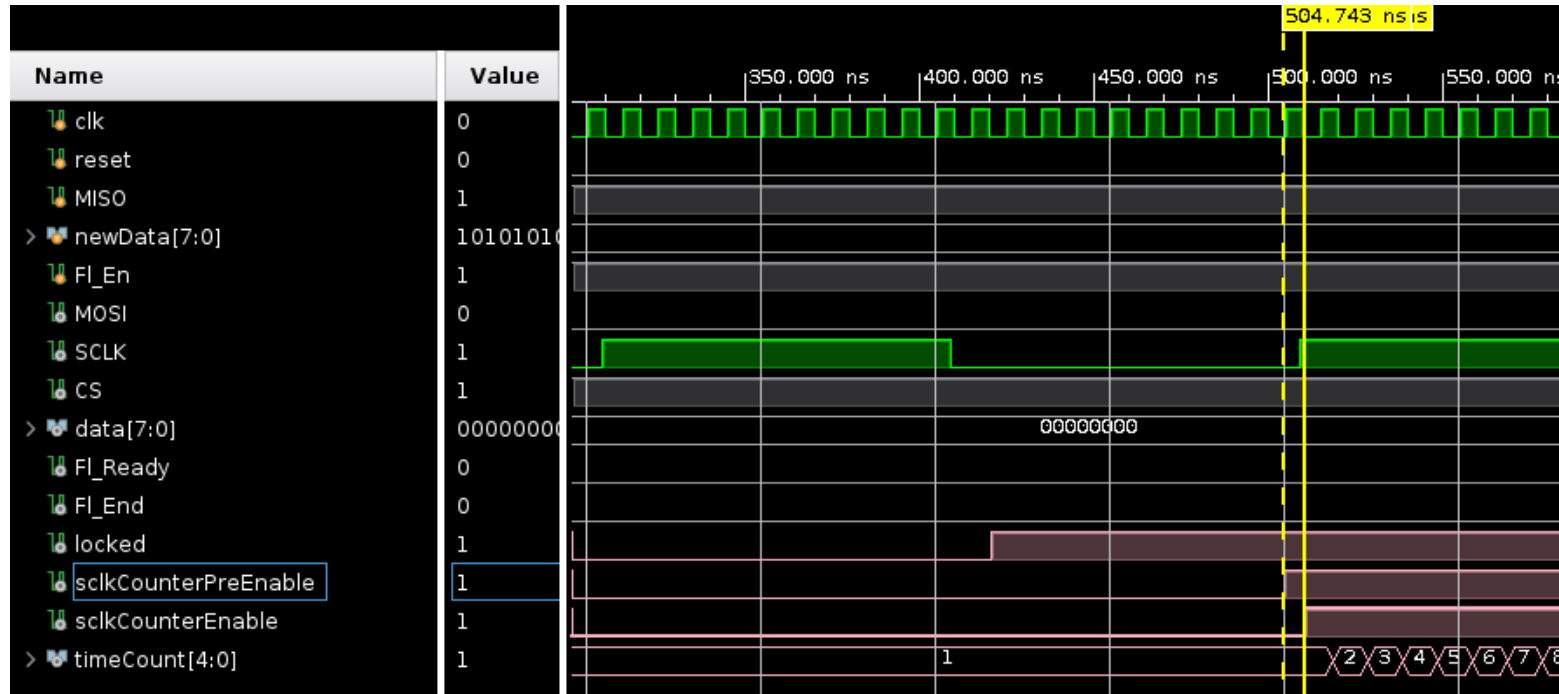
Έπειτα πρέπει να επαληθεύσουμε ότι λειτουργούν τα σήματα σωστά (πχ mosi, miso κτλ.) για μια κανονική επικοινωνία με τη χρήση της μονάδας

Μπορούμε να ελένξουμε τη σωστή λειτουργία σε περίπτωση που το FI_En ανεβεί στη μέση της επικοινωνίας.

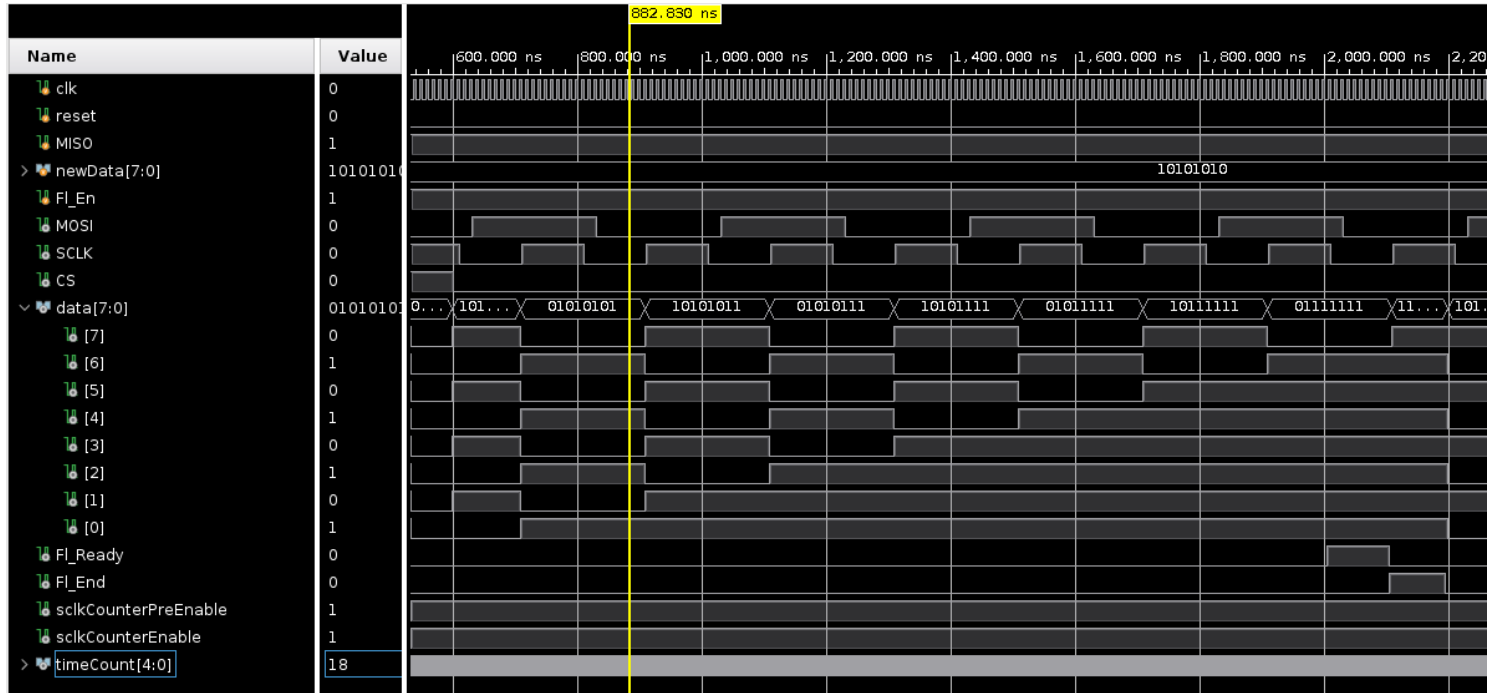
Η δοκιμή περιλαμβάνει τους παραπάνω ελέγχους για αποστολή του 10101010 και τη λήψη του 11111111.

Παρακάτω φαίνονται στιγμιότυπα της παραπάνω δοκιμής για post-implementation timing simulation.

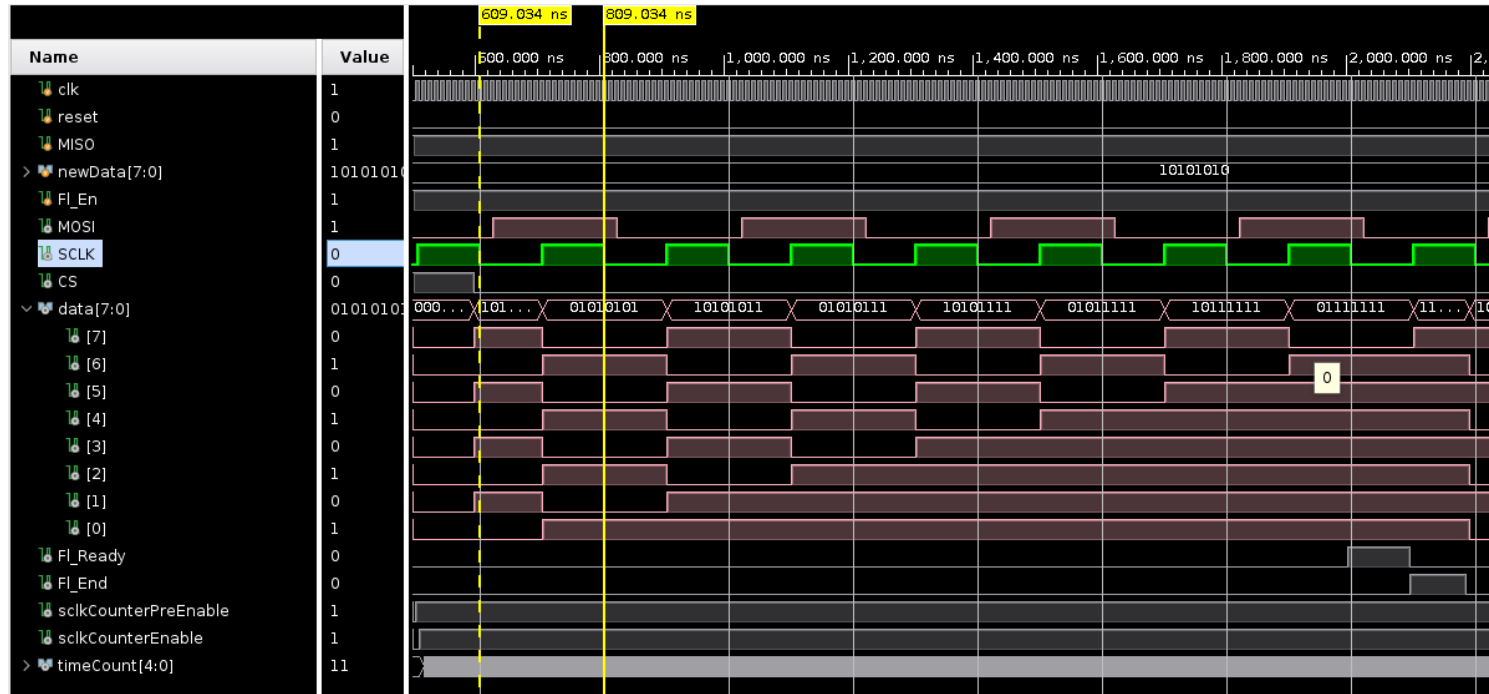
SCLK_Synchronisation



Byte Transmission Example



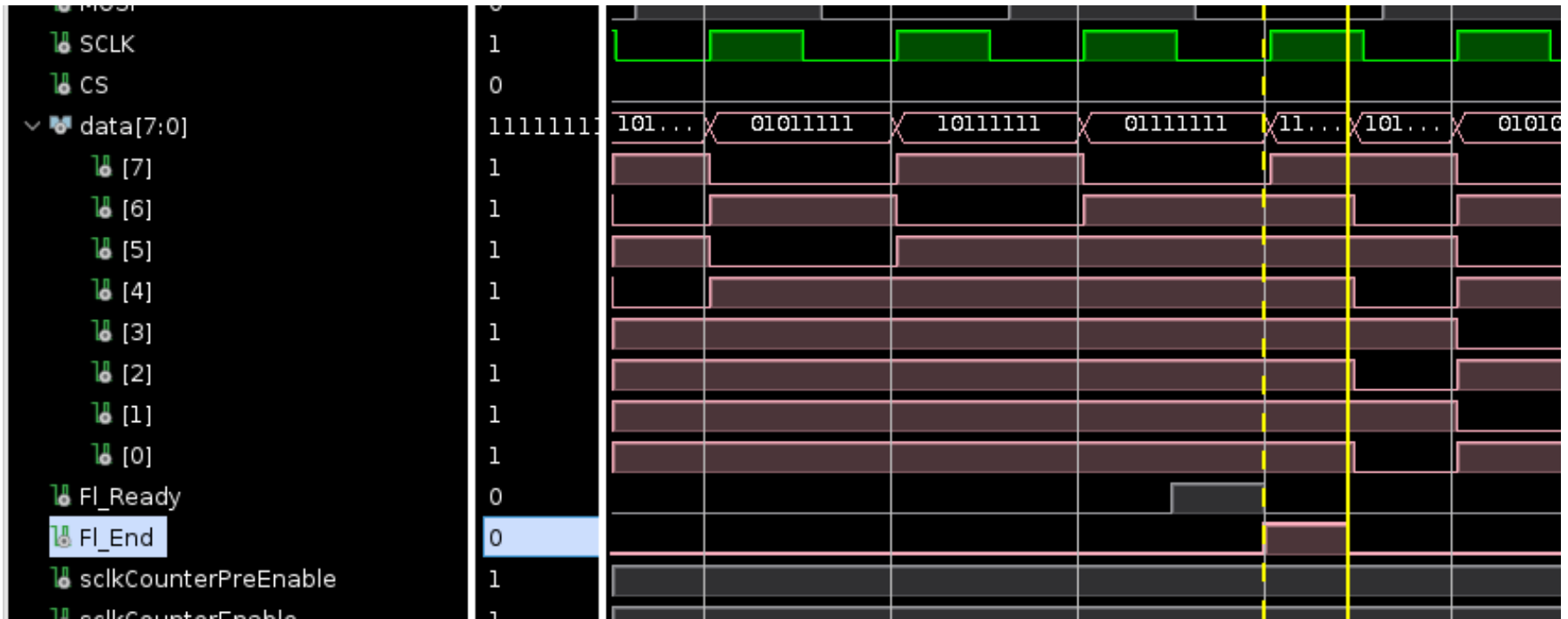
Byte Transmission Example: MOSI



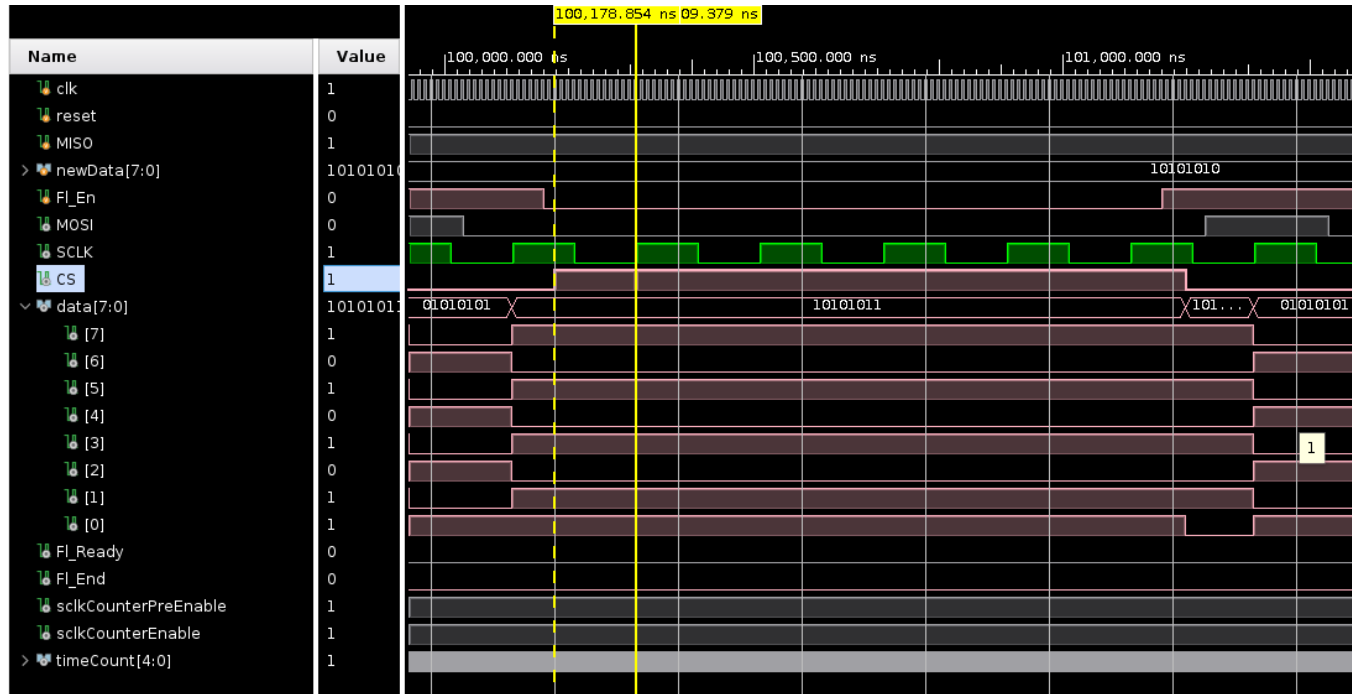
Byte Transmission Example: MISO



Byte Transmission Example: End Of Transmission

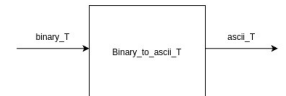
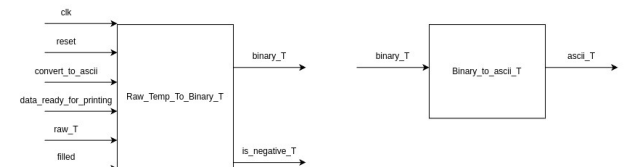
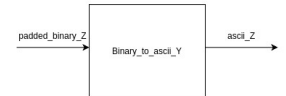
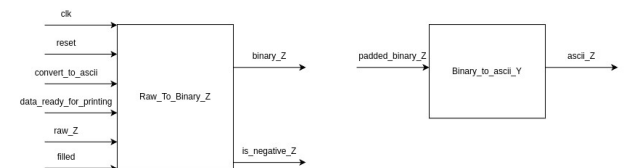
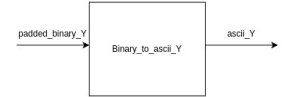
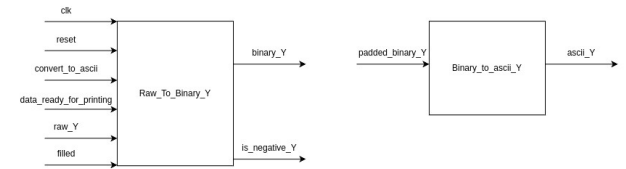
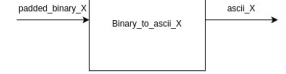
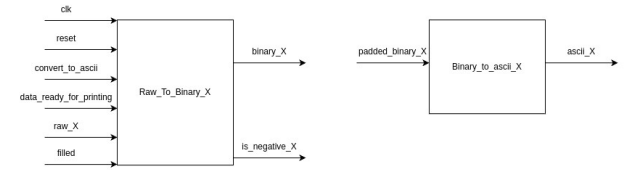
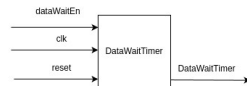
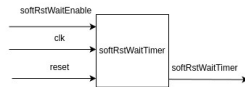
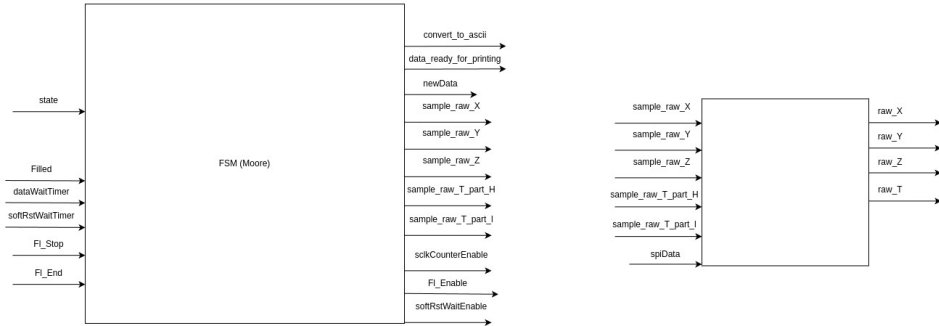
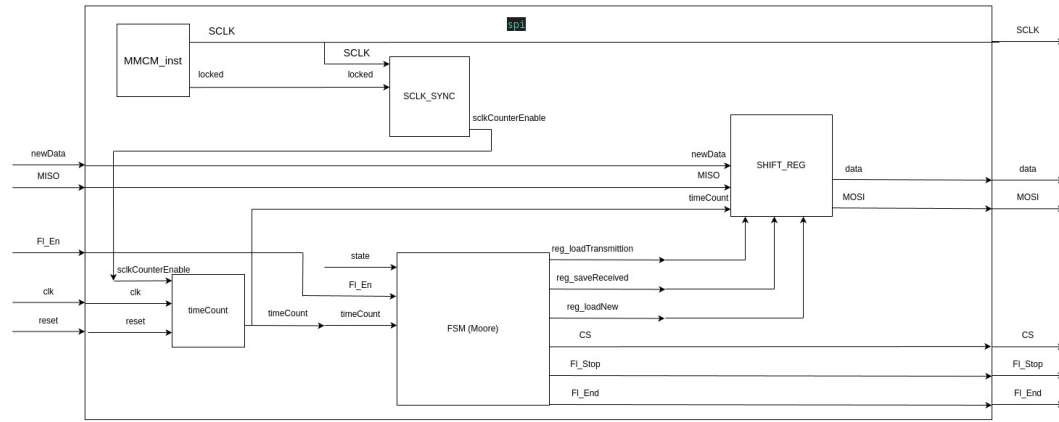


Byte Transmission Example: Transmission Stop



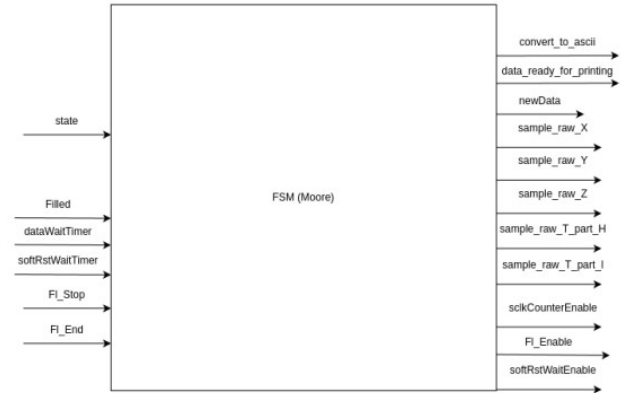
3ο Μέρος: Υλοποίηση Ελεγκτή ADXL362

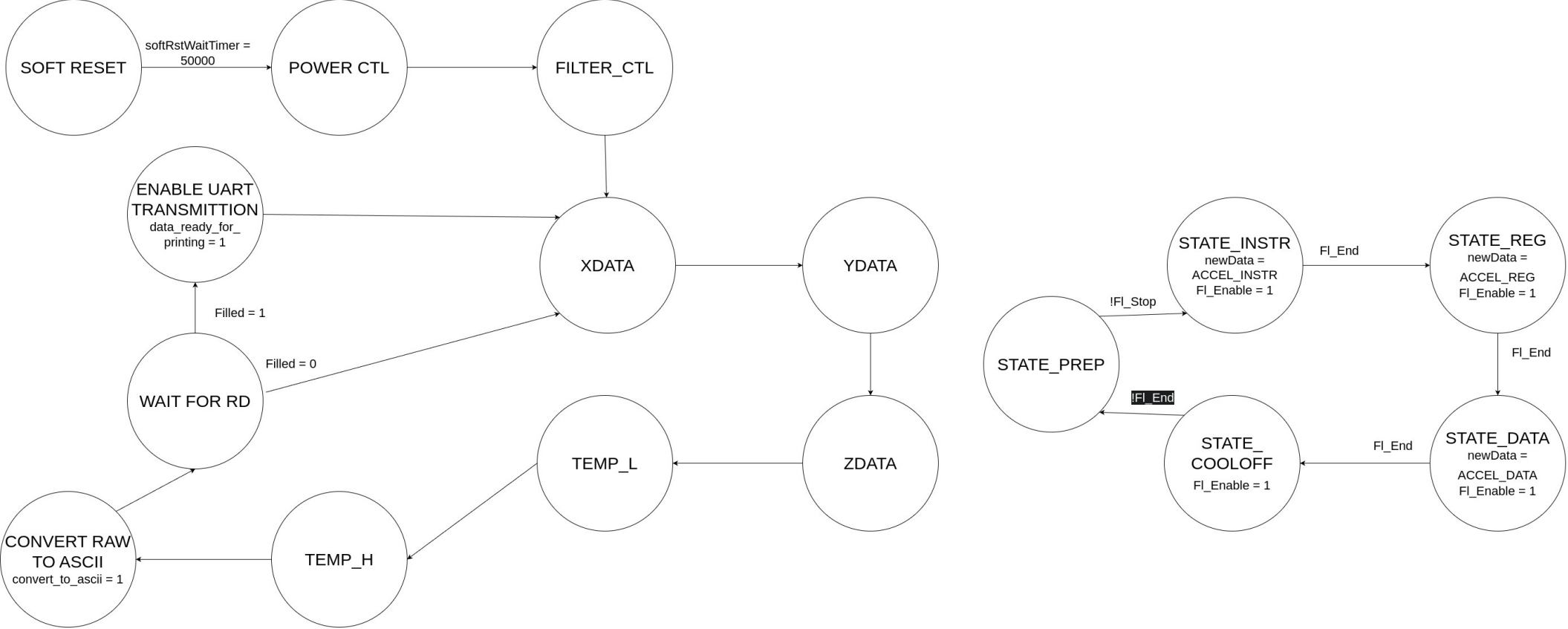
- Σε αυτό το κομμάτι θα προγραμματίσουμε την επικοινωνία με το αξελερόμετρο.
- Στην επόμενη διαφάνεια φαίνεται το διάγραμμα ροής.



FSM

- Η σωστή ανάθεση των δεδομένων, ή καλύτερα η ανάθεση αυτών την σωστή χρονική στιγμή γίνεται με την χρήση FSM.
- Η FSM είναι τύπου moore και υλοποιήθηκε με τη χρήση 2 always block, ενός συνδιαστικού που θα είναι υπεύθυνο για τις εξόδους της (προφανώς αυτό περιλαμβάνει και την ανάθεση του επόμενου state), και ενός ακολουθιακού, που αναθέτει συνεχώς στο state το επόμενο state (που αναφέρθηκε προηγουμένως).
- Στις επόμενες διαφάνειες φαίνονται αναλυτικότερα τα state που περιέχει και την λειτουργία του καθενός απο αυτά, η καλύτερα την δομή που υιοθετούν για την σωστή λειτουργία της μονάδας.





Τα περισσότερα state που φαίνονται αριστερά είναι συμβολικά, και η πλήρης δομή τους θα αναλυθεί στις επόμενες διαφάνειες

SPI Timeline

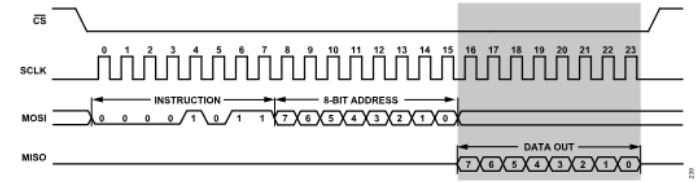


Figure 36. Register Read

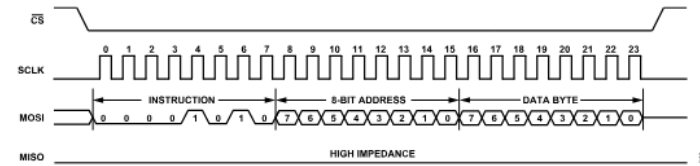
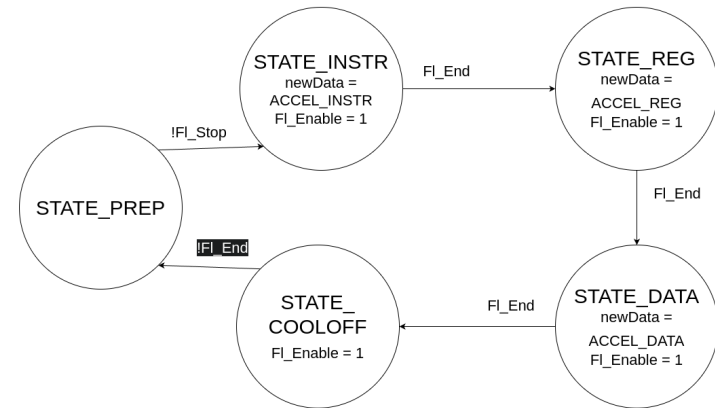


Figure 37. Register Write (Receive Instruction Only)



SPI Command Timeline

- Αρχικά θα σταλθεί η εντολή (ανάγνωση/εγγραφή, ή και fifo που δεν θα χρησιμοποιηθεί)
- Λόγω της υλοποίησης του spi master, θα μεταβαίνουμε συνήθως στο επόμενο state εναν κύκλο πριν τελειώσει η αναμετάδοση. Αυτό είναι εσκεμμένα, για να προλάβουμε να αναθέσουμε νέα δεδομένα στον spi master (την επόμενη εντολή)

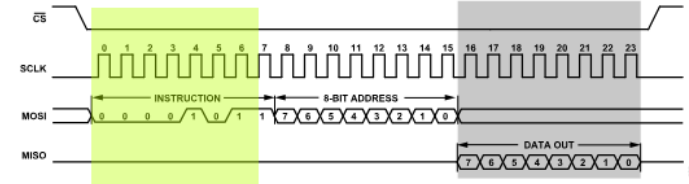


Figure 36. Register Read

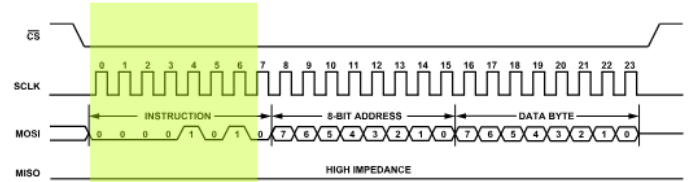
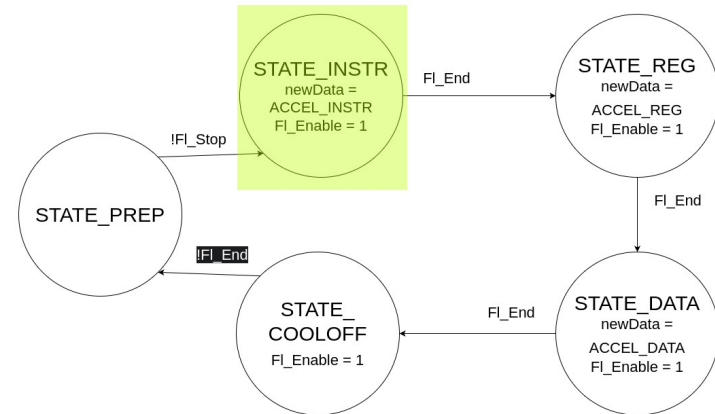


Figure 37. Register Write (Receive Instruction Only)



SPI Command Timeline

- Σε αυτό το state θα στείλουμε την τοποθεσία του καταχωρητή στον οποίο θέλουμε να γράψουμε/απο τον οποίο θέλουμε να διαβάσουμε

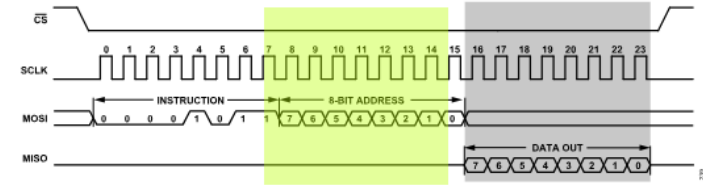


Figure 36. Register Read

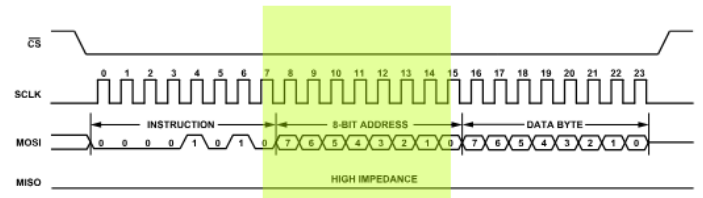
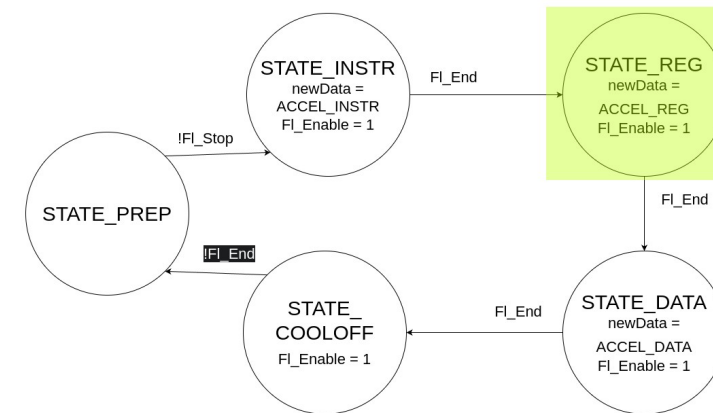


Figure 37. Register Write (Receive Instruction Only)



SPI Command Timeline

- Εδώ θα στείλουμε τα δεδομένα, ή θα περιμένουμε την λήψη τους.

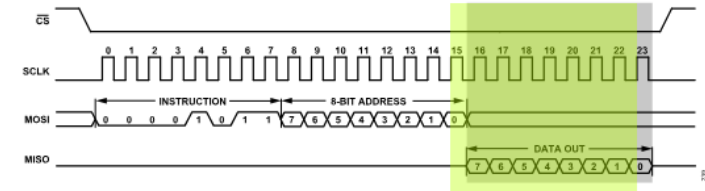


Figure 36. Register Read

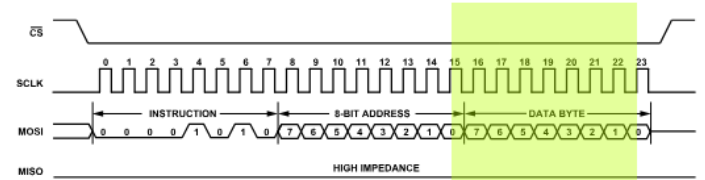
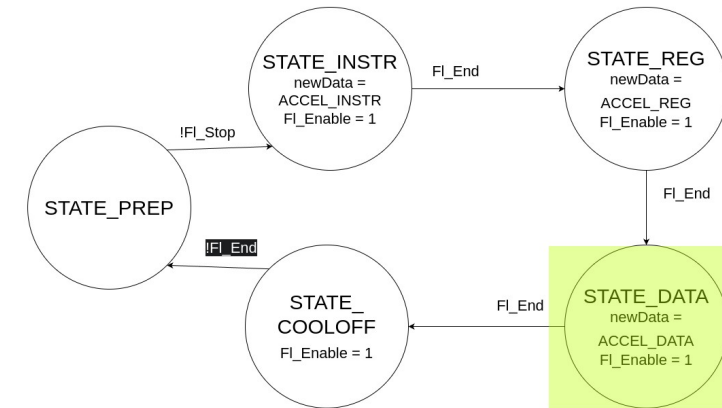
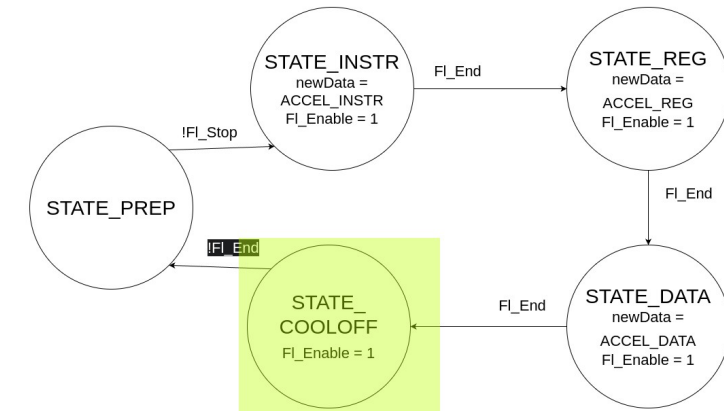
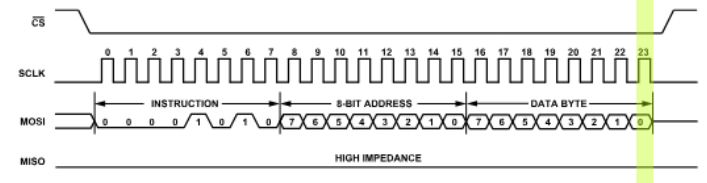
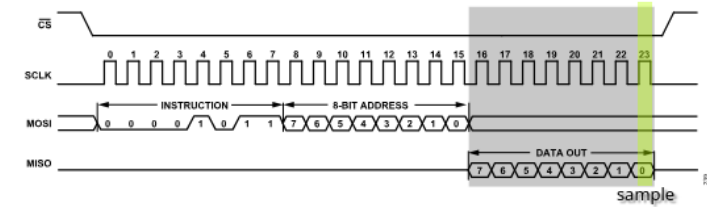


Figure 37. Register Write (Receive Instruction Only)



SPI Command Timeline

- Σε αυτό το state θα περιμένουμε να τελειώσει πλήρως η επικοινωνία, για την αποφυγή παραβίασης κάποιου χρονισμού.
- Σε περίπτωση που ζητήσαμε την ανάγνωση των δεδομένων, εδώ είναι η κατάλληλη στιγμή να τα κάνουμε δειγματοληψία, καθώς αυτή είναι η μόνη στιγμή που το spi master έχει λάβει όλα τα δεδομένα. Σε αυτή τη περίπτωση το state θα ονομάζεται συμβολικά STATE_DATA_SAMPLE.



SPI Command Timeline

- Στο τέλος της αναμετάδωσης, θα περιμένουμε να τερματίσει ομαλά η επικοινωνία

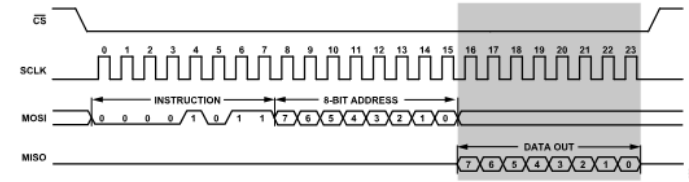


Figure 36. Register Read

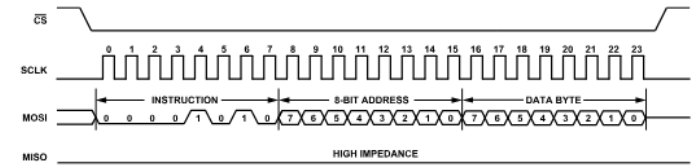
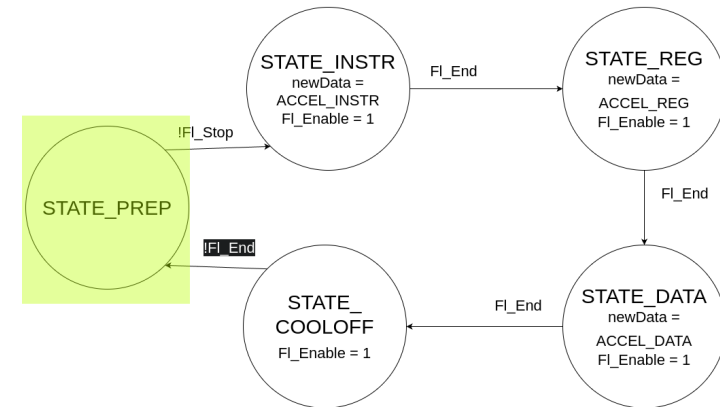


Figure 37. Register Write (Receive Instruction Only)



SPI Command Timeline: Burst Read

- Το επιταχυνσιόμετρο υποστηρίζει burst read: αν δεν ανεβεί το σήμα CS, θα συνεχίσει να στέλνει τα δεδομένα του επόμενου καταχωρητή.
- Σε περίπτωση που το χρησιμοποιήσουμε, θα εναλασσόμαστε μεταξύ STATE_DATA και STATE_DATA_SAMPLE για κάθε καινούριο δεδομένο που λαμβάνουμε, μέχρι να τερματίσουμε την επικοινωνία

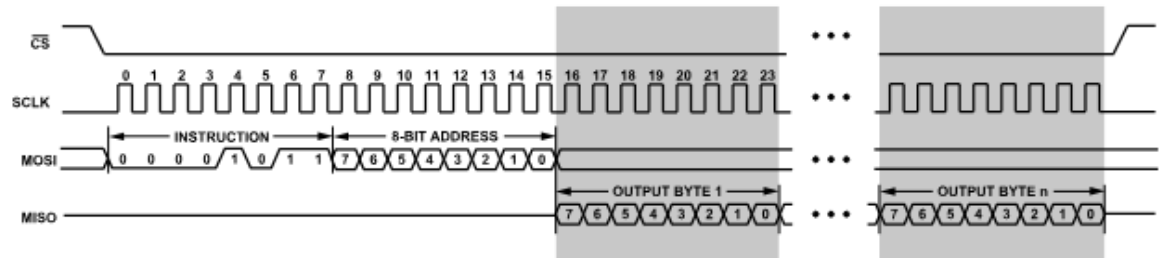
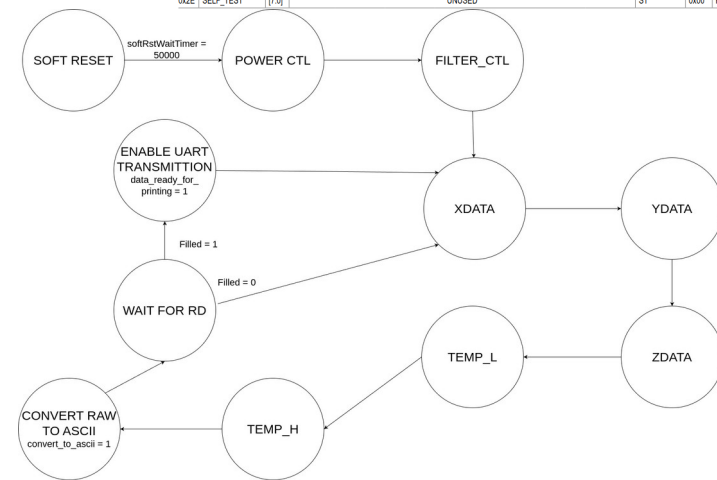


Figure 38. Burst Read

Communication Timeline

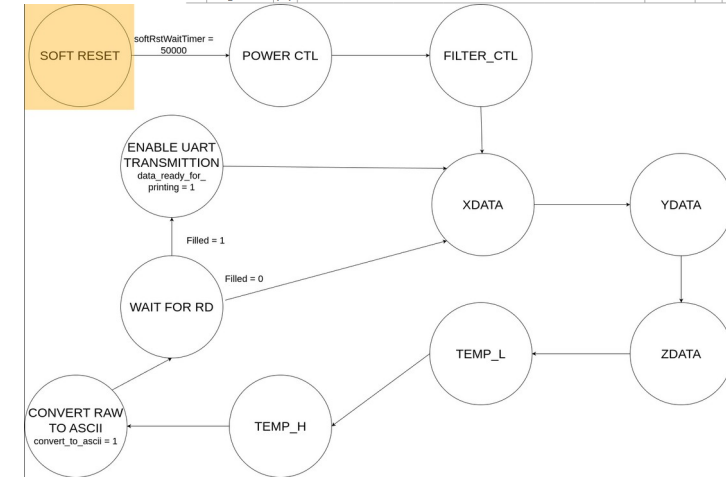
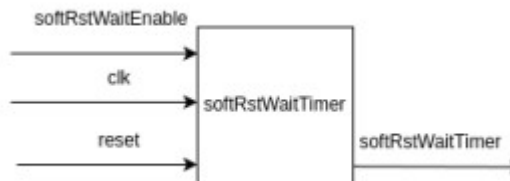
Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW	
0x00	DEVID_AD	[7:0]				DEVID_AD[7:0]				0xAD	R	
0x01	DEVID_MST	[7:0]				DEVID_MST[7:0]				0x1D	R	
0x02	PARTID	[7:0]				PARTID[7:0]				0xF2	R	
0x03	REVID	[7:0]				REVID[7:0]				0x01	R	
0x08	XDATA	[7:0]				XDATA[7:0]				0x00	R	
0x09	YDATA	[7:0]				YDATA[7:0]				0x00	R	
0x0A	ZDATA	[7:0]				ZDATA[7:0]				0x00	R	
0x0E	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R_MARK	FIFO_READ Y	DATA_READ Y	0x40	R
0x0C	FIFO_ENTRIES_ _L	[7:0]				FIFO_ENTRIES_ _L[7:0]				0x00	R	
0x0D	FIFO_ENTRIES_ _H	[7:0]			UNUSED			FIFO_ENTRIES_ _H[7:0]		0x00	R	
0x0E	XDATA_L	[7:0]				XDATA_L[7:0]				0x00	R	
0x0F	XDATA_H	[7:0]		SK				XDATA_H[3:0]		0x00	R	
0x10	YDATA_L	[7:0]				YDATA_L[7:0]				0x00	R	
0x11	YDATA_H	[7:0]		SK				YDATA_H[3:0]		0x00	R	
0x12	ZDATA_L	[7:0]				ZDATA_L[7:0]				0x00	R	
0x13	ZDATA_H	[7:0]		SK				ZDATA_H[3:0]		0x00	R	
0x14	TEMP_L	[7:0]				TEMP_L[7:0]				0x00	R	
0x15	TEMP_H	[7:0]		SK				TEMP_H[3:0]		0x00	R	
0x16	Reserved	[7:0]				Reserved[7:0]				0x00	R	
0x17	Reserved	[7:0]				Reserved[7:0]				0x00	R	
0x1F	SOFT_RESET	[7:0]				SOFT_RESET[7:0]				0x00	W	
0x20	THRESH_ACT_L	[7:0]				THRESH_ACT_L[7:0]				0x00	RW	
0x21	THRESH_ACT_H	[7:0]			UNUSED			THRESH_ACT_H[2:0]		0x00	RW	
0x22	TIME_ACT	[7:0]				TIME_ACT[7:0]				0x00	RW	
0x23	THRESH_NACT_ _L	[7:0]				THRESH_NACT_ _L[7:0]				0x00	RW	
0x24	THRESH_NACT_ _H	[7:0]			UNUSED			THRESH_NACT_H[2:0]		0x00	RW	
0x25	TIME_NACT_L	[7:0]				TIME_NACT_L[7:0]				0x00	RW	
0x26	TIME_NACT_H	[7:0]				TIME_NACT_H[7:0]				0x00	RW	
0x27	ACT_NACT_CTL	[7:0]	RES		LINKLOOP	INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW	
0x28	FIFO_CONTROL	[7:0]			UNUSED	AH	FIFO_TEMP	FIFO_MODE		0x00	RW	
0x29	FIFO_SAMPLES	[7:0]				FIFO_SAMPLES[7:0]				0x00	RW	
0x2A	INTMP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]		RANGE	RES	HALF_BW	EXT_SAMP L_E		OOD	0x13	RW	
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK		LOW_NOISE	WAKEUP	AUTOSLEEP		MEASURE	0x00	RW
0x2E	SELF_TEST	[7:0]				UNUSED			ST		0x00	RW



Communication Timeline: Soft Reset

- Αρχικά θα χρειαστεί να κάνουμε reset το επιταχυνσιόμετρο.
- Μετά το reset, θα περιμένουμε 0.5ms (όπως ορίζεται απο το manual).
- Θα υλοποιήσουμε έναν μετρητή σε ένα ξεχωριστό always block.

Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0x4D	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x1D	R
0x02	PARTID	[7:0]					PARTID[7:0]				0x2F	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R-MARK	FIFO_READ Y	DATA_READ Y	0x40	R
0x0C	FIFO_ENTRIES_L	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_H	[7:0]					UNUSED		FIFO_ENTRIES_H[1:0]		0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]		SX				XDATA_H[3:0]			0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]		SX				YDATA_H[3:0]			0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]		SX				ZDATA_H[3:0]			0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]		SX				TEMP_H[3:0]			0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]		UNUSED				THRESH_ACT_H[2:0]			0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_L	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_H	[7:0]		UNUSED				THRESH_INACT_H[2:0]			0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]	RES		LINKLOOP		INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW
0x28	FIFO_CONTROL	[7:0]		UNUSED			AH	FIFO_TEMP		FIFO_MODE	0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x80	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]		RANGE	RES	HALF_BW	EXT_SAMPL E		ODR		0x13	RW
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK		LOW_NOISE	WAKEUP	AUTOSLEEP		MEASURE ST	0x00	RW
0x2E	SELF_TEST	[7:0]					UNUSED				0x00	RW

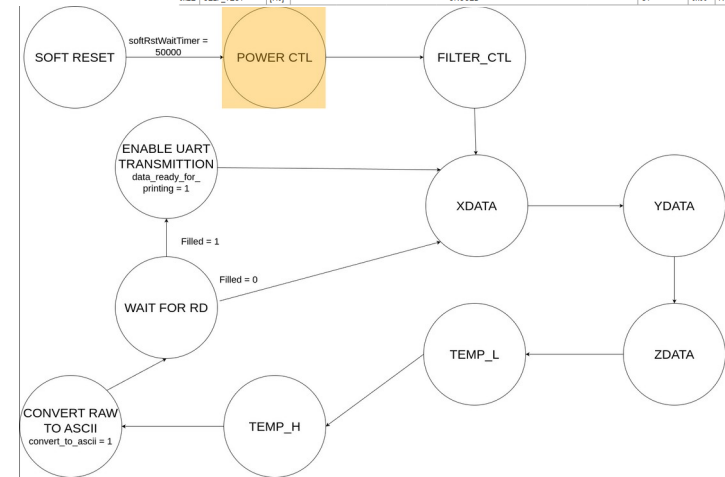


Communication Timeline: Enable Measurement

Θα στείλουμε την εντολή 00000010:

- Reserved: x (0)
- Ext_clk: internal clock(0)
- Low noise: normal operation (00, reset default)
- Wakeup: Measurement mode (0)
- Autosleep: Disabled (0)
- Measure: Measurement mode (10)

Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0xAD	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x10	R
0x02	PARTID	[7:0]					PARTID[7:0]				0xF2	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R-MARK	FIFO_READ Y	DATA_READ Y	0x40	R
0x0C	FIFO_ENTRIES_L	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_H	[7:0]					UNUSED		FIFO_ENTRIES_H[1:0]		0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]			SX			XDATA_H[3:0]			0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]			SX			YDATA_H[3:0]			0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]			SX			ZDATA_H[3:0]			0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]			SX			TEMP_H[3:0]			0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]			UNUSED			THRESH_ACT_H[2:0]			0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_L	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_H	[7:0]			UNUSED			THRESH_INACT_H[2:0]			0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]	RES		LINKLOOP		INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW
0x28	FIFO_CONTROL	[7:0]			UNUSED		AH	FIFO_TEMP		FIFO_MODE	0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x80	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WATE R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]		RANGE	RES	HALF_BW	EXT_SAMPL E		ODR		0x13	RW
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK		LOW_NOISE	WAKEUP	AUTOSLEEP		MEASURE	0x00	RW
0x2E	SELF_TEST	[7:0]					UNUSED			ST	0x00	RW

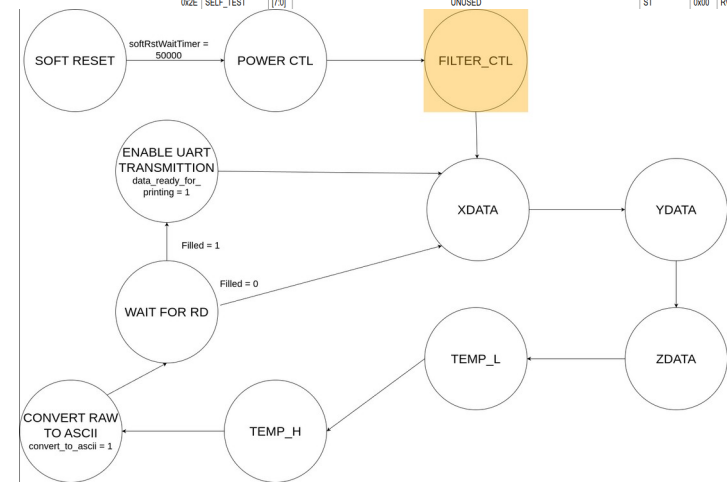


Communication Timeline: Configure Filter

Θα στείλουμε την εντολή 00010100:

- Range: 2g (00)
- Reserved: x (0)
- Half_bw: 1/4 bandwidth (1)
- Ext_Sample: disable (0)
- ORD: 200Hz (100)

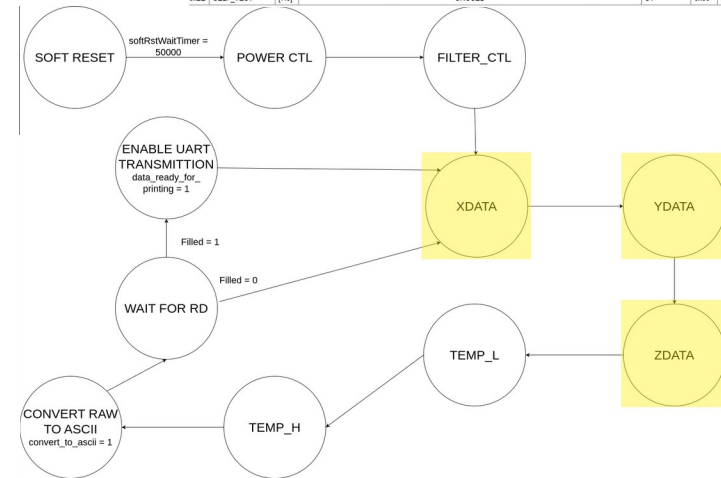
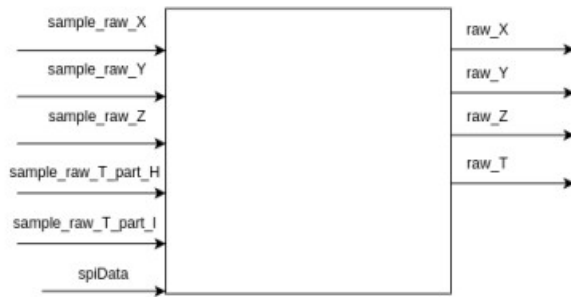
Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0xAd	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x1D	R
0x02	PARTID	[7:0]					PARTID[7:0]				0xF2	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVERFLOW -RUN	FIFO_WAT R-MARK	FIFO_READ Y	DATA_READ Y	0x4D	R
0x0C	FIFO_ENTRIES_L	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_H	[7:0]					UNUSED			FIFO_ENTRIES_H[1:0]	0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]		SX				XDATA_H[3:0]			0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]		SX				YDATA_H[3:0]			0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]		SX				ZDATA_H[3:0]			0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]		SX				TEMP_H[3:0]			0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]					UNUSED			THRESH_ACT_H[2:0]	0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_L	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_H	[7:0]					UNUSED			THRESH_INACT_H[2:0]	0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]	RES		UNLOOP		INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW
0x28	FIFO_CONTROL	[7:0]			UNUSED		AH	FIFO_TEMP		FIFO_MODE	0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x80	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVERFLOW -RUN	FIFO_WAT R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVERFLOW -RUN	FIFO_WAT R-MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]	RANGE	RES	HALF_BW	EXT_SAMPL E			ORD		0x13	RW
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK	LOW_NOISE	WAKEUP		AUTOSLEEP		MEASURE	0x00	RW
0x2E	SELF_TEST	[7:0]					UNUSED			ST	0x00	RW



Communication Timeline: Retrieve XYZ Data

- Σε αυτά τα state, η fsm θα σηματοδοτήσει την αποθήκευση των δεδομένων που μας αποστέλλει το spi σε αντίστοιχους καταχωρητές.
- Λόγω της θέσης των καταχωρητών, μπορούμε να κάνουμε χρήση της λειτουργίας burst read που μας προσφέρει το επιταχυνσιόμετρο.

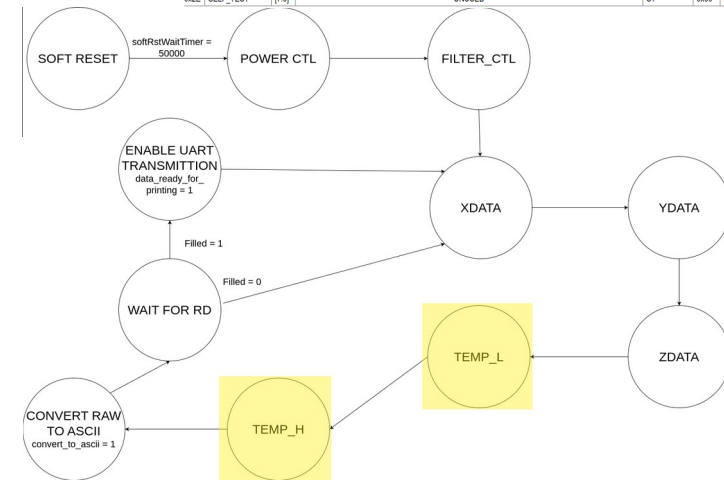
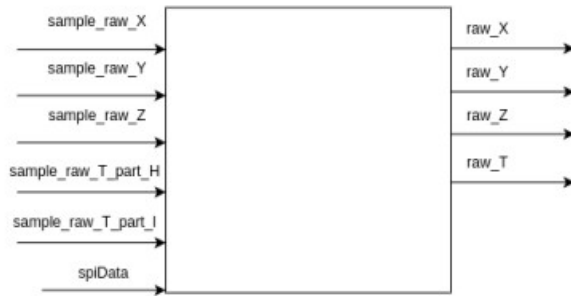
Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0xAD	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x10	R
0x02	PARTID	[7:0]					PARTID[7:0]				0xF2	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WIATE R_MARK	FIFO_READ Y	DATA_READ Y	0x40	R
0x0C	FIFO_ENTRIES_	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_	[7:0]					UNUSED		FIFO_ENTRIES_H[1:0]		0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]			SX				XDATA_H[3:0]		0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]			SX				YDATA_H[3:0]		0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]			SX				ZDATA_H[3:0]		0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]			SX				TEMP_H[3:0]		0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]					UNUSED		THRESH_ACT_H[2:0]		0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_	[7:0]					UNUSED		THRESH_INACT_H[2:0]		0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]		RES		UNLOOP	INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW
0x28	FIFO_CONTROL	[7:0]					UNUSED	AH	FIFO_TEMP	FIFO_MODE	0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x80	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WIATE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WIATE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]		RANGE	RES	HALF_BW	EXT_SAMPL E		ODR		0x13	RW
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK		LOW_NOISE	WAKEUP	AUTOSLEEP		MEASURE ST	0x00	RW
0x2E	SELF_TEST	[7:0]					UNUSED				0x00	RW



Communication Timeline: Retrieve Temperatures

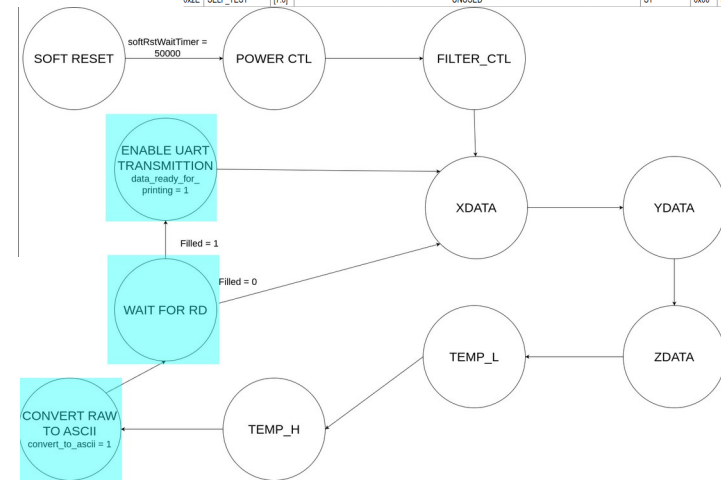
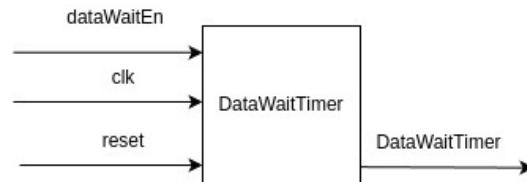
- Θα λειτουργήσουμε όμοια με προηγούμενω, μόνο που αυτή τη φορά θα λάβουμε μια 12-bit μεταβλητή στον χρόνο δύο λήψεων.
- Όπως και πριν, μπορούμε να λάβουμε τα δεδομένα απο τον δεύτερο καταχωρητή μέσω burst

Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0xAD	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x1D	R
0x02	PARTID	[7:0]					PARTID[7:0]				0xF2	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]	ERR_USE R_REGS	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R_MARK	FIFO_READ Y	DATA_READ Y	0x4D	R
0x0C	FIFO_ENTRIES_L	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_H	[7:0]					UNUSED		FIFO_ENTRIES_H[1:0]		0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]			SX			XDATA_H[3:0]			0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]			SX			YDATA_H[3:0]			0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]			SX			ZDATA_H[3:0]			0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]			SX			TEMP_H[3:0]			0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]			UNUSED			THRESH_ACT_H[2:0]			0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_L	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_H	[7:0]			UNUSED			THRESH_INACT_H[2:0]			0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]	RES		LINKLOOP		INACT_REF	INACT_EN	ACT_REF	ACT_EN	0x00	RW
0x28	FIFO_CONTROL	[7:0]			UNUSED		AH	FIFO_TEMP		FIFO_MODE	0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x80	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER -RUN	FIFO_WAITE R_MARK	FIFO_READ Y	DATA_READ Y	0x00	RW
0x2C	FILTER_CTL	[7:0]		RANGE	RES	HALF_BW	EXT_SAMPL E		ODR		0x13	RW
0x2D	POWER_CTL	[7:0]	RES		EXT_CLK	LOW_NOISE	WAKEUP	AUTOSLEEP		MEASURE ST	0x00	RW
0x2E	SELF_TEST	[7:0]					UNUSED				0x00	RW



Communication Timeline: Sample/Transmit Data

- Τα state αυτά δεν είναι συμβολικά και δέν περιλαμβάνουν εντολές προς το αξελερόμετρο.
- Αφορούν την σωστή οργάνωση της δειγματοληψίας και της αποστολής των δεδομένων.
- Το state CONVERT RAW TO ASCII θα επιτρέψει τα δεδομένα που συλλέχθηκαν να κρατηθούν για να γίνει εντέλει η μετατροπή τους σε binary και έπειτα σε ascii.
- Παράλληλα, ένας μετρητής δεδομένων (dataCount) θα αυξηθεί κατά 1. Όταν μετρήσει 128 δεδομένα (σηματοδοτείται διαμέσω του FILLED), τότε θα βγάλουμε τον μέσο όρο απο δεδομένα που συλλέχτηκαν και θα μεταβούμε στη μετάδοση τους. Ο έλεγχος αυτός θα γίνει στο WAIT FOR RD state.
- Πρίν την μετάβαση στο ENABLE UART TRANSMISSION STATE, σε περίπτωση που έχουν μετρηθεί αρκετά δεδομένα, θα περιμένουμε μερικούς κύκλους στο WAIT FOR RD state για να γίνει σωστά η μετατροπή τους. Περισσότερες λεπτομέρειες για την μετατροπή παρέχονται παρακάτω



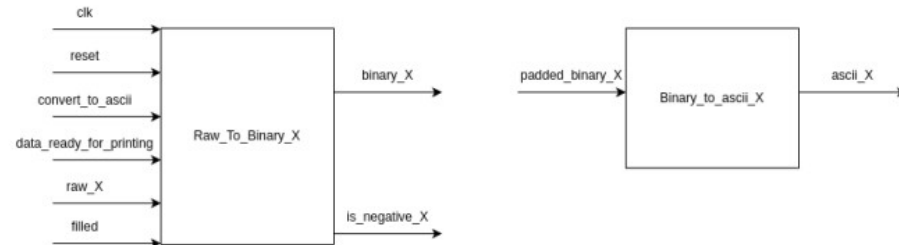
Reg	Name	Bits	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset	RW
0x00	DEVID_AD	[7:0]					DEVID_AD[7:0]				0xAd	R
0x01	DEVID_MST	[7:0]					DEVID_MST[7:0]				0x1D	R
0x02	PARTID	[7:0]					PARTID[7:0]				0xF2	R
0x03	REVID	[7:0]					REVID[7:0]				0x01	R
0x08	XDATA	[7:0]					XDATA[7:0]				0x00	R
0x09	YDATA	[7:0]					YDATA[7:0]				0x00	R
0x0A	ZDATA	[7:0]					ZDATA[7:0]				0x00	R
0x0B	STATUS	[7:0]					STATUS[7:0]				0x00	R
0x0C	FIFO_ENTRIES_L	[7:0]					FIFO_ENTRIES_L[7:0]				0x00	R
0x0D	FIFO_ENTRIES_H	[7:0]					FIFO_ENTRIES_H[7:0]				0x00	R
0x0E	XDATA_L	[7:0]					XDATA_L[7:0]				0x00	R
0x0F	XDATA_H	[7:0]					XDATA_H[7:0]				0x00	R
0x10	YDATA_L	[7:0]					YDATA_L[7:0]				0x00	R
0x11	YDATA_H	[7:0]					YDATA_H[7:0]				0x00	R
0x12	ZDATA_L	[7:0]					ZDATA_L[7:0]				0x00	R
0x13	ZDATA_H	[7:0]					ZDATA_H[7:0]				0x00	R
0x14	TEMP_L	[7:0]					TEMP_L[7:0]				0x00	R
0x15	TEMP_H	[7:0]					TEMP_H[7:0]				0x00	R
0x16	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x17	Reserved	[7:0]					Reserved[7:0]				0x00	R
0x1F	SOFT_RESET	[7:0]					SOFT_RESET[7:0]				0x00	W
0x20	THRESH_ACT_L	[7:0]					THRESH_ACT_L[7:0]				0x00	RW
0x21	THRESH_ACT_H	[7:0]					THRESH_ACT_H[7:0]				0x00	RW
0x22	TIME_ACT	[7:0]					TIME_ACT[7:0]				0x00	RW
0x23	THRESH_INACT_L	[7:0]					THRESH_INACT_L[7:0]				0x00	RW
0x24	THRESH_INACT_H	[7:0]					THRESH_INACT_H[7:0]				0x00	RW
0x25	TIME_INACT_L	[7:0]					TIME_INACT_L[7:0]				0x00	RW
0x26	TIME_INACT_H	[7:0]					TIME_INACT_H[7:0]				0x00	RW
0x27	ACT_INACT_CTL	[7:0]					ACT_INACT_CTL[7:0]				0x00	RW
0x28	FIFO_CONTROL	[7:0]					FIFO_CONTROL[7:0]				0x00	RW
0x29	FIFO_SAMPLES	[7:0]					FIFO_SAMPLES[7:0]				0x00	RW
0x2A	INTMAP1	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER	FIFO_WAT	FIFO_READ	DATA_READ	0x00	RW
0x2B	INTMAP2	[7:0]	INT_LOW	AWAKE	INACT	ACT	FIFO_OVER	FIFO_WAT	FIFO_READ	DATA_READ	0x00	RW
0x2C	FILTER_CTL	[7:0]					FILTER_CTL[7:0]				0x13	RW
0x2D	POWER_CTL	[7:0]	RES	EXT_CLK	LOW_NOISE	WAKEUP	AUTOSLEEP	MEASURE			0x00	RW
0x2E	SELF_TEST	[7:0]					SELF_TEST[7:0]				0x00	RW

Μετατροπή Raw Σε Binary

Για την χρήση των δεδομένων που λήφθηκαν, θα χρειαστεί να τα μετατρέψουμε σε τιμές binary, και έπειτα σε ascii. Για την μετατροπή των raw τιμών σε binary, θα ακολουθήσουμε τα εξής βήματα:

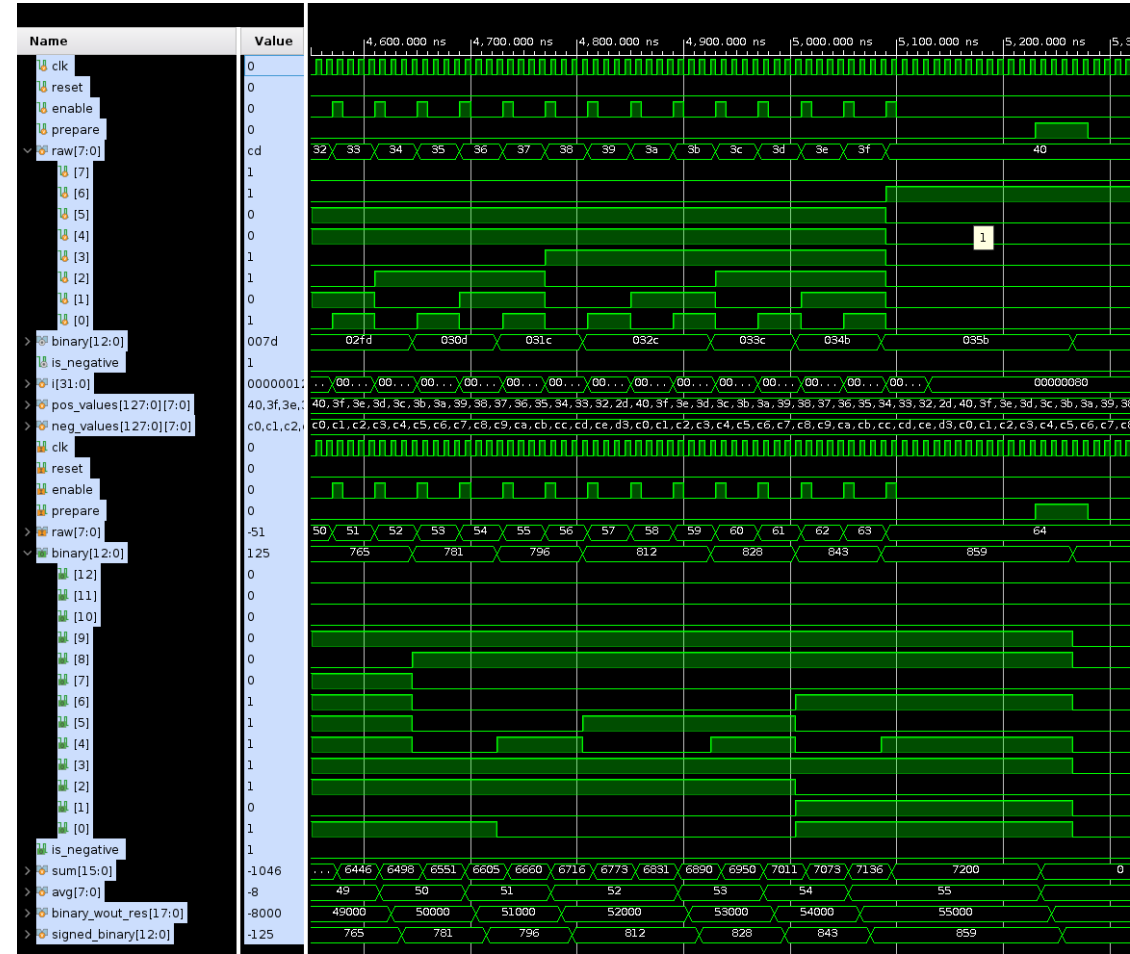
- Θα πάρουμε τον μέσο όρο 128 τιμών (το οποίο δίνει χρόνο στο uart για να προλάβει να στείλει τα προηγούμενα δεδομένα)
 - Θα τις πολλαπλασιάσουμε με 1000, ώστε να είναι σε milig. Η εκτέλεση αυτού του βήματος πρώτου θα μας αποτρέψει απο το να χάσουμε ακρίβεια απο την επόμενη διαίρεση.
 - Θα πολ/σουμε το αποτέλεσμα με Scaling Factor = Full-Scale range / Maximum Raw value = $4g(+2g)/2^8(=256) = 1/64$
 - → Για τη θερμοκρασία, γνωρίζουμε απο το manual οτι πρέπει να πολ/στεί με 0.065
 - Βρίσκουμε το πρόσημο και το αναθέτουμε στον is_negative, έπειτα το αφαιρούμε απο την αρχική μεταβλητή
- Το σωστό αποτέλεσμα θα εμφανιστεί σαν έξοδος μόνο αφού έχουν ληφθεί και οι 128 τιμές, για να είναι σταθερή η μετάδοση μέσω UART.

Επίσης για την αποφυγή timing violation, τα παραπάνω βήματα είναι pipelined μέσω flip flop. Αυτό σημαίνει πως το αποτέλεσμα θα βγεί μετά απο κάποιους κύκλους καθυστέρησης.

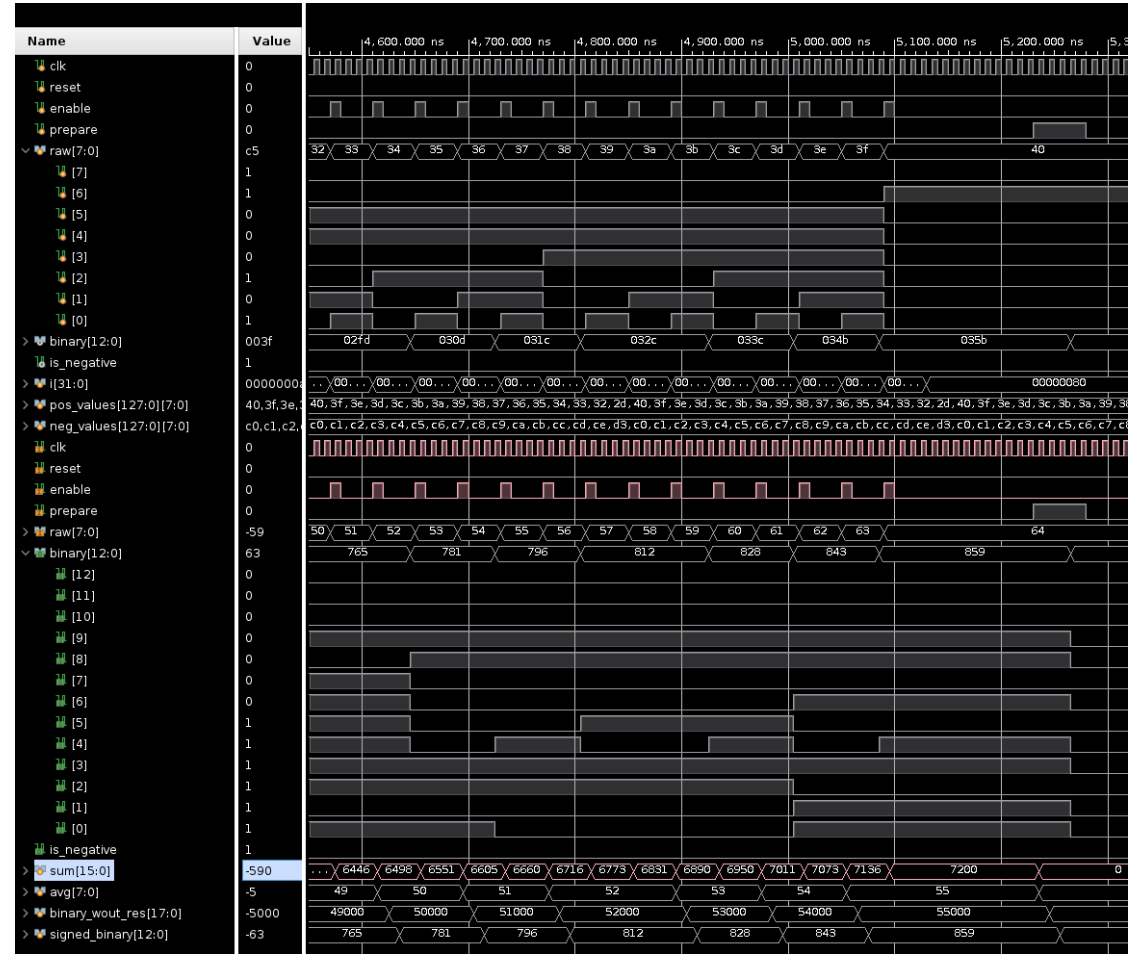


Προσωμοίωση Μετατροπής Raw Σε Binary

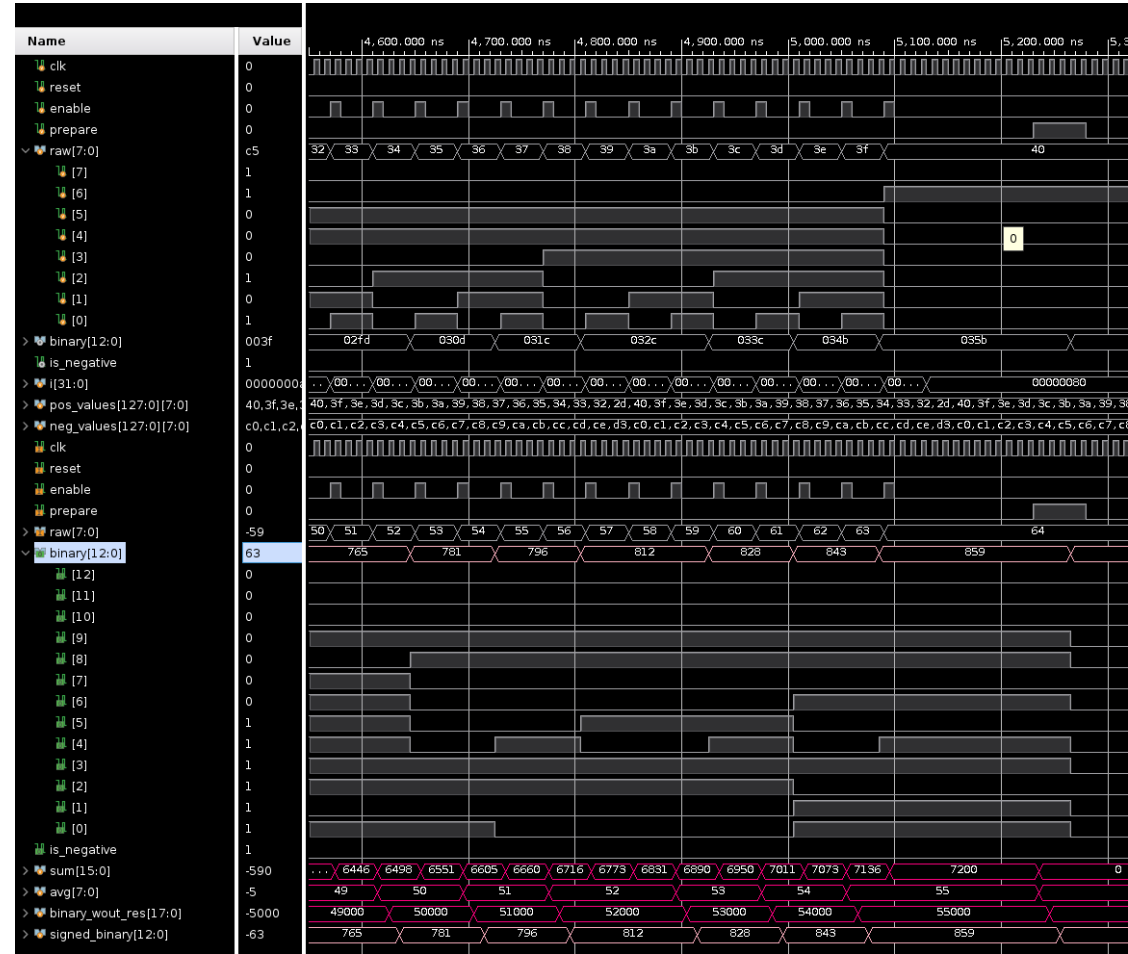
- Για να φαίνονται καλύτερα τα βήματα στη μετατροπή, η προσωμοίωση είναι αρχικά behavioural



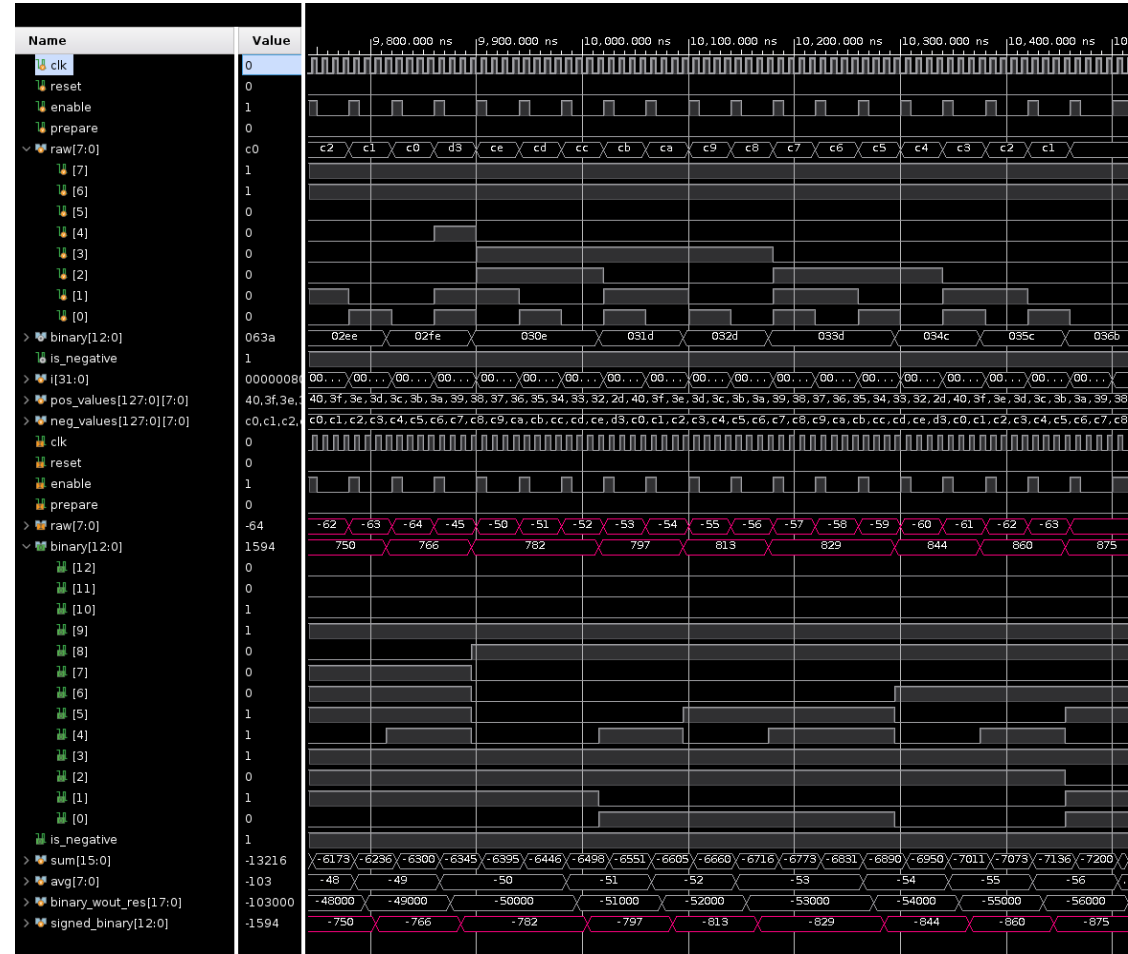
Προσωμοίωση Μετατροπής Raw Σε Binary



Προσωμοίωση Μετατροπής Raw Σε Binary

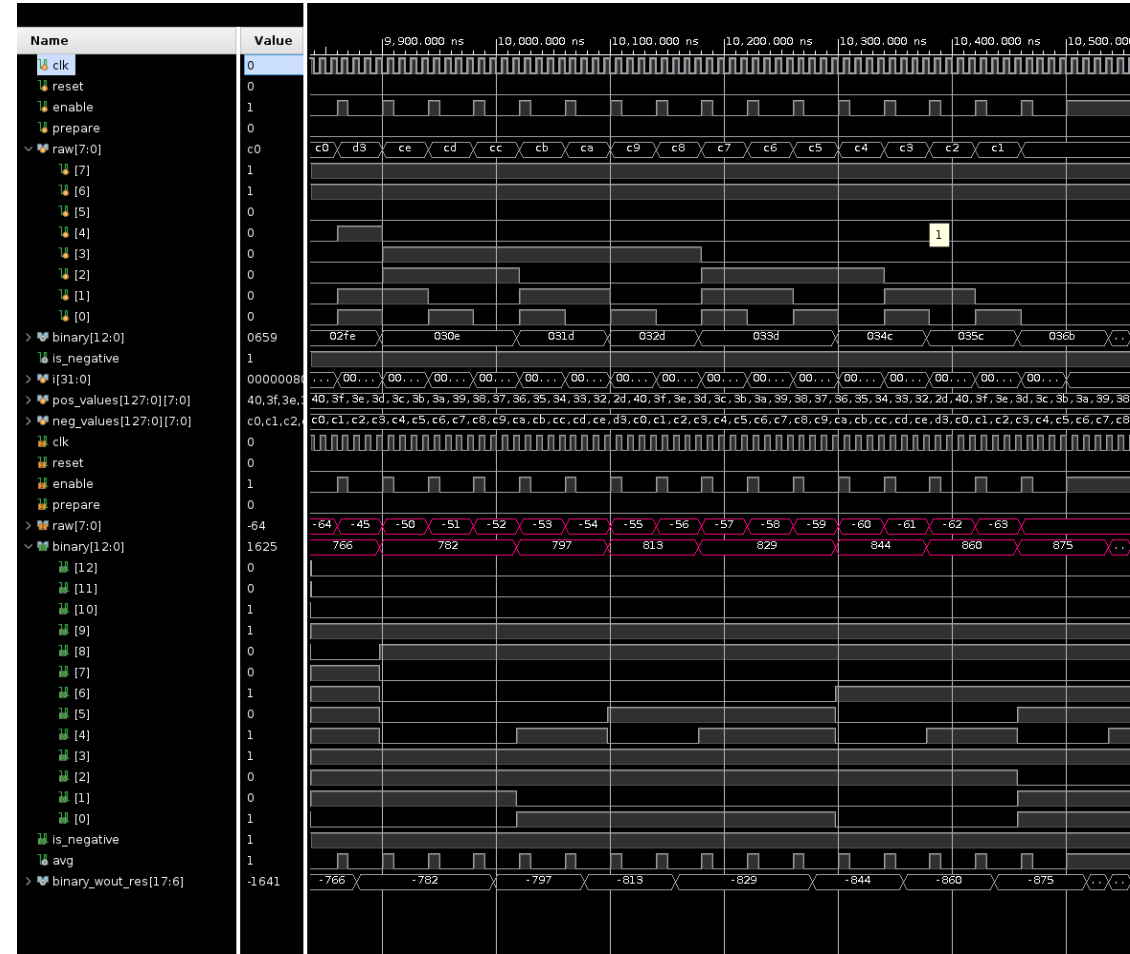


Προσωμοίωση Μετατροπής Raw Σε Binary



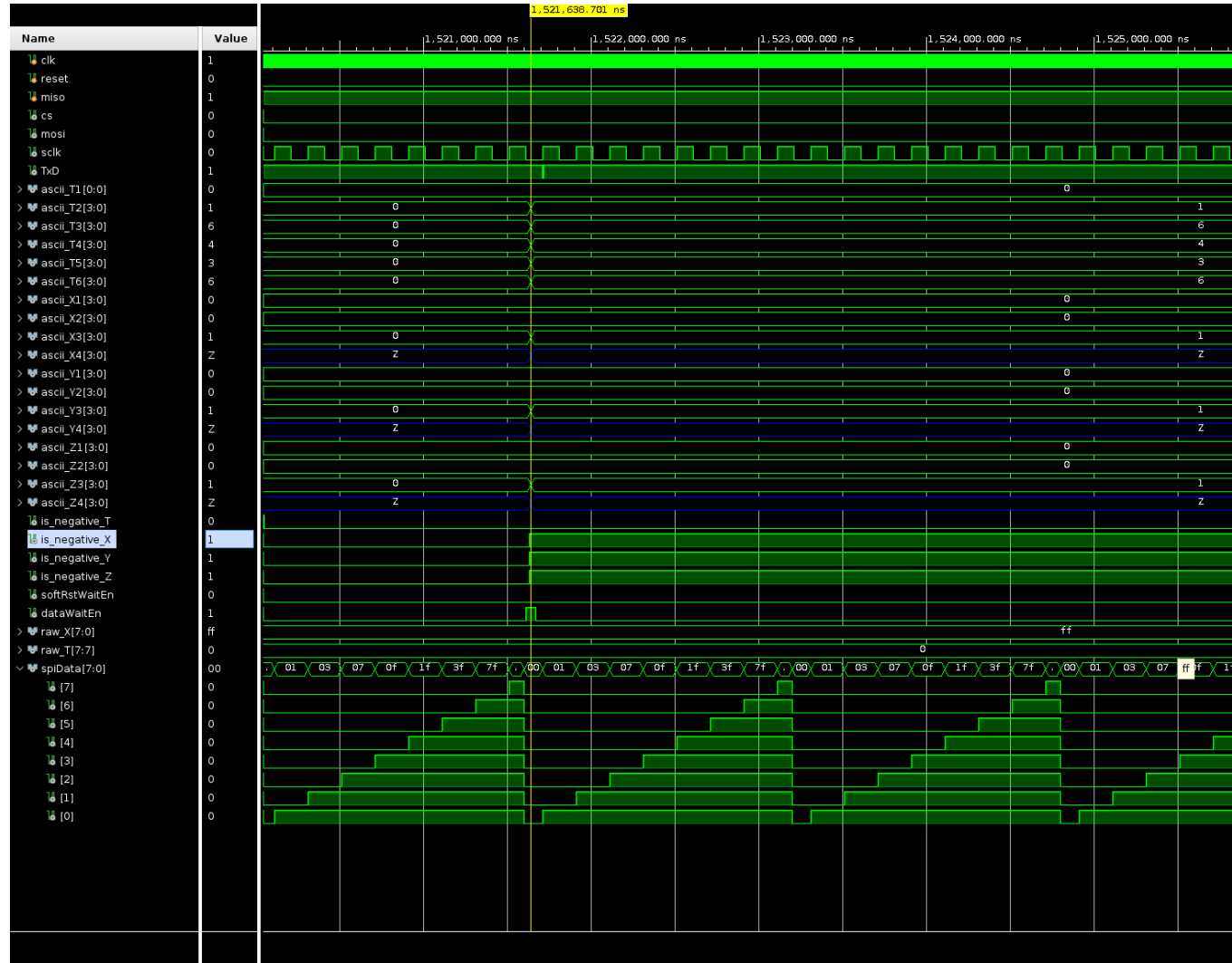
Προσωμοίωση Μετατροπής Raw Σε Binary

- Εδώ φαίνεται η σωστή λειτουργία σε post implementation

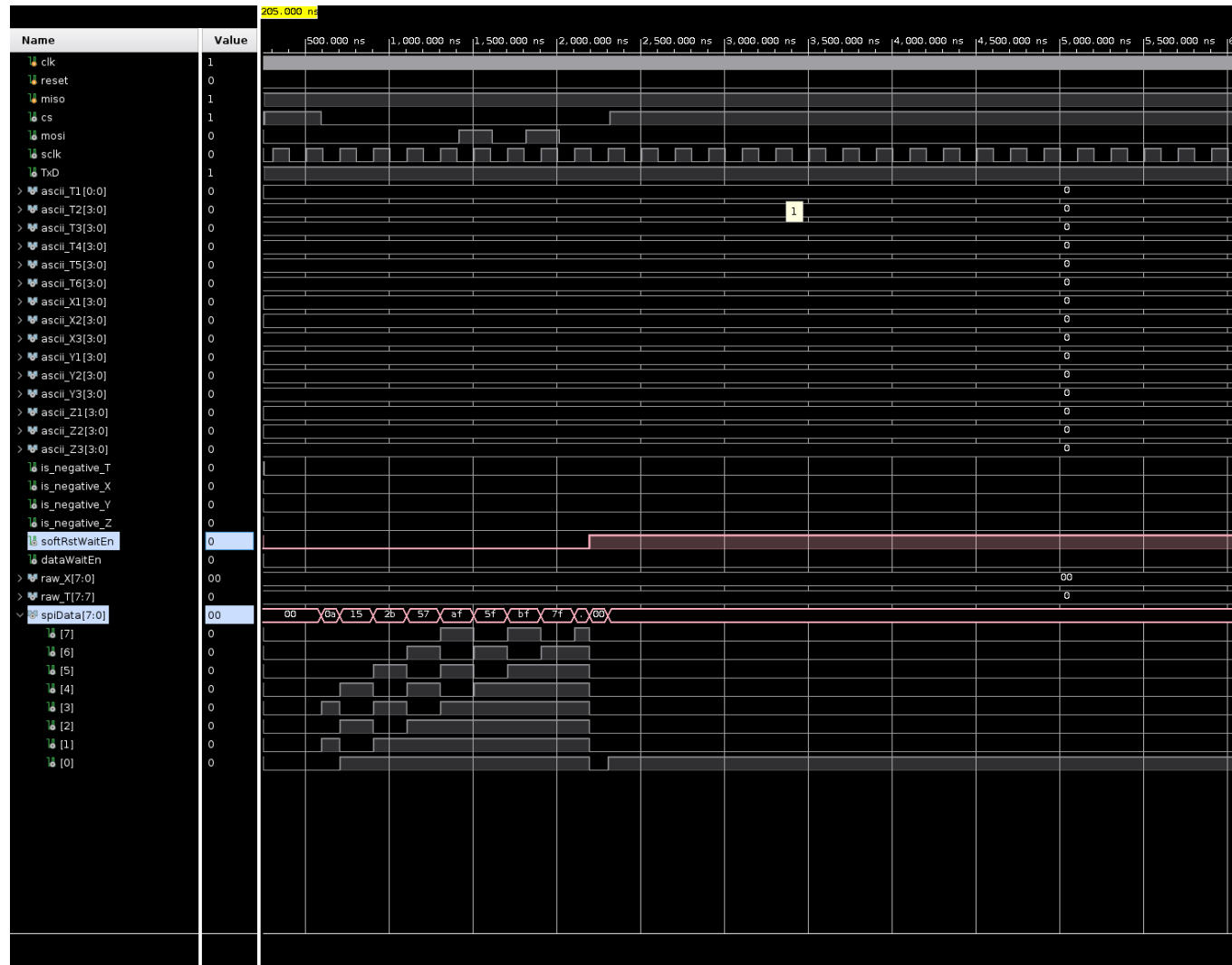


Προσωμοίωση Accelerometer Driver

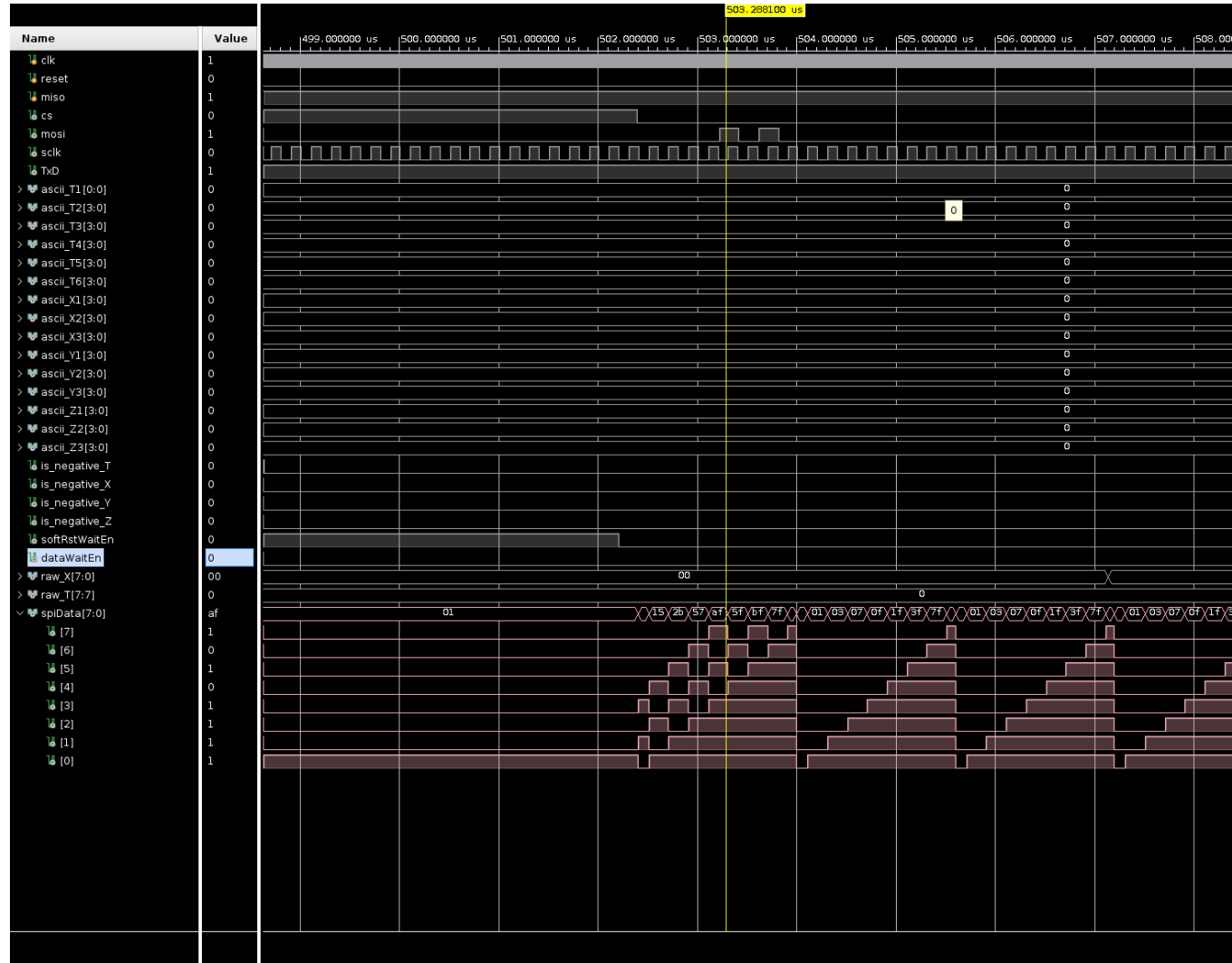
- Οι κυματομορφές φαίνονται σε post-implementation



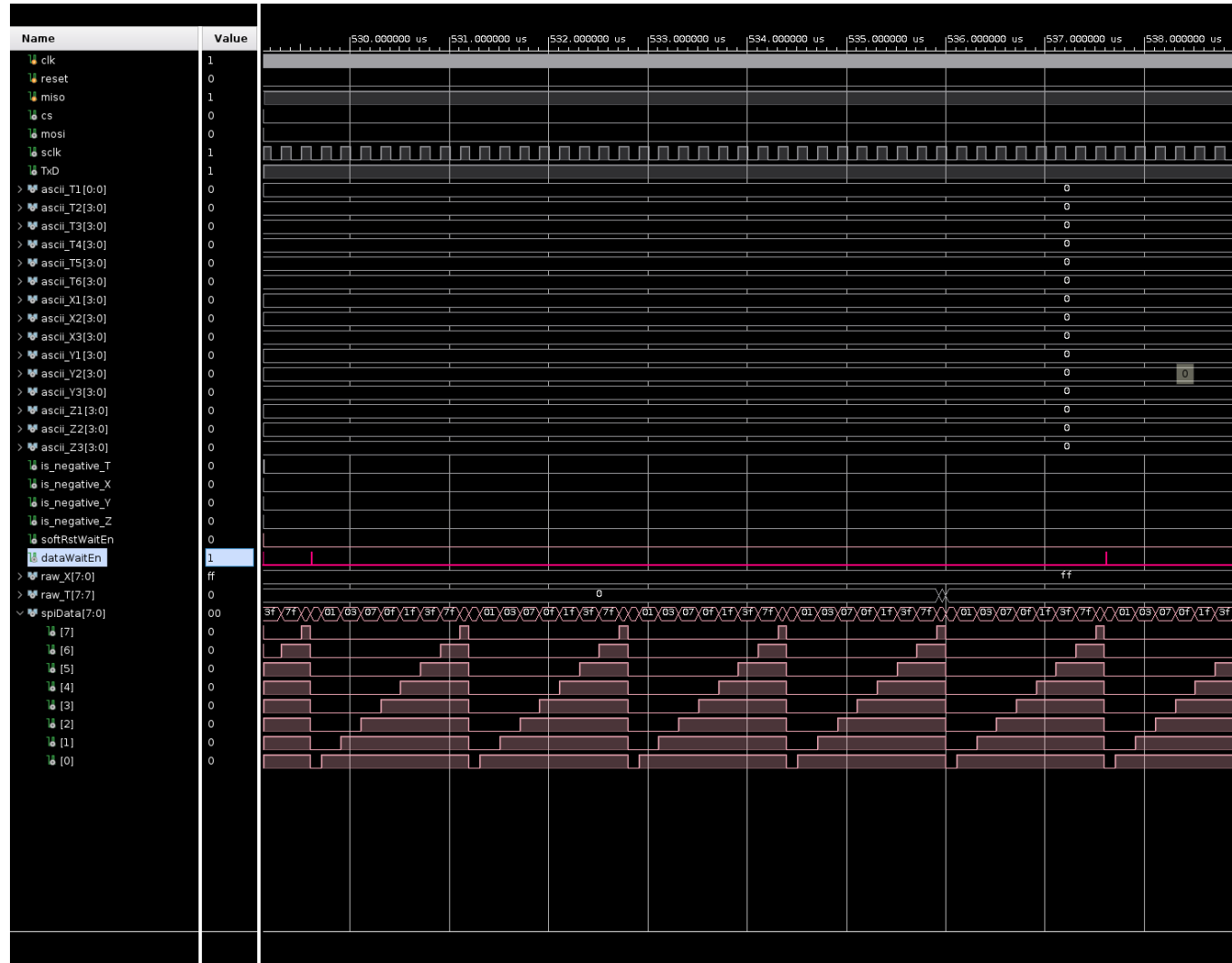
Προσωμοίωση Accelerometer Driver



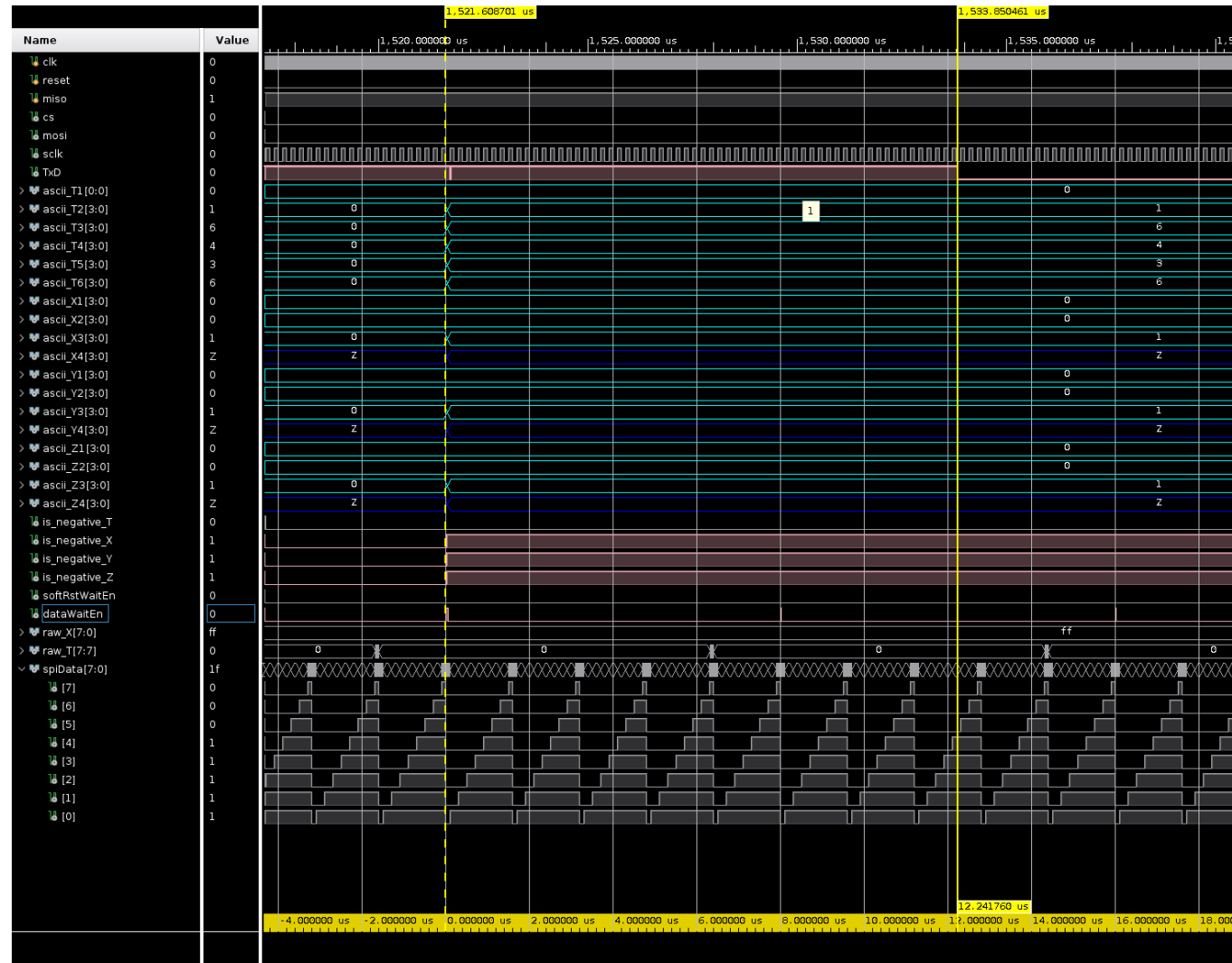
Προσωμοίωση Accelerometer Driver



Προσωμοίωση Accelerometer Driver



Προσωμοίωση Accelerometer Driver



Ευχαριστώ για την προσοχή σας!